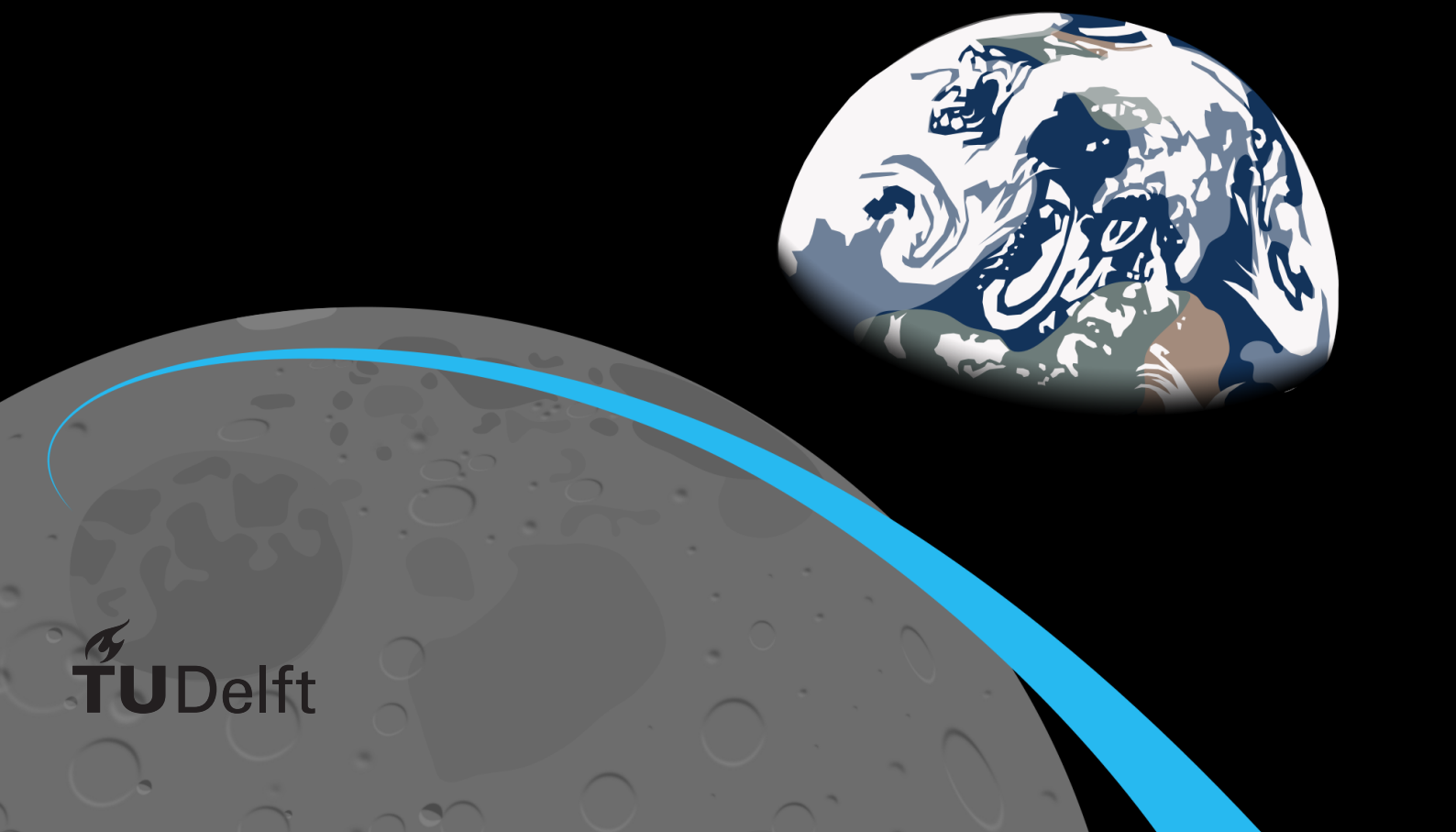
Orbital ascent trajectory optimization using reinforcement learning

Preliminary thesis

A. M. de Windt



TU Delft

Orbital ascent trajectory optimization using reinforcement learning

Preliminary thesis

by

A. M. de Windt

to obtain the degree of Master of Science
at the Delft University of Technology,
to be defended publicly on Tuesday January 1, 2013 at 10:00 AM.

Student number:	1234567	
Project duration:	March 1, 2012 – January 1, 2013	
Thesis committee:	Prof. dr. ir. J. Doe,	TU Delft, supervisor
	Dr. E. L. Brown,	TU Delft
	Ir. A. Aaronson,	Acme Corporation

This thesis is confidential and cannot be made public until December 31, 2013.

An electronic version of this thesis is available at `http://repository.tudelft.nl/`.

TU Delft

Todo list

Remove this citation, it's needed so latex doesn't crash. 2
Rewrite this once chapter is done 3
Rewite this paragraph. 18
Add missing figure referenced by this paragraph 18
Rewrite this paragraph. 20
Add reference to the transition probability function explanation in previous chapter. 20
Fix the tuple 24
Update captions 24

Preface

Preface…

A. M. de Windt
Delft, January 2013

List of Figures

2.1 Powered flight trajectory[11]. 6
2.2 Possible variation of V 9
2.3 Optimal $V(t)$ for different options for Φ[11]. 10
2.4 Gravity turn[11]. 11
2.5 Optimal control closed and open loop solutions. 13

3.1 Example reward distribution for a 5-armed bandit problem. 20
3.2 Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies. 21
3.3 Example reward distribution for a 5-armed bandit problem. 25
3.4 Example reward distribution for a 5-armed bandit problem. 25
3.5 Artificial Neural Network[10]. 28
3.6 Minorize-Maximization algorithm 32

List of Tables

Symbols

G_t return (cumulative discounted reward) following time t.

$G_t^{\lambda a}$ λ-return, corrected by estimated action values.

$G_t^{\lambda s}$ λ-return, corrected by estimated state values.

$G_{t:h}$ flat return (uncorrected, undiscounted) from $t+1$ to h.

$G_{t:h}^{\lambda a}$ truncated, corrected λ-return, with action values.

$G_{t:h}^{\lambda s}$ truncated, corrected λ-return, with state values.

Q, Q_t array estimates of action-value function q_π or q_*.

V, V_t array estimates of state-value function v_π or v_*.

$\in$ is an element of, e.g., $s \in \mathcal{S}$.

$|\mathcal{S}|$ number of elements in $\mathcal{S}$.

$\mathcal{A}$ set of all actions.

$\mathcal{R}$ set of all possible rewards, a finite subset of $\mathbb{R}$.

$\mathcal{S}$ set of all states.

$\mathcal{S}^+$ set of all states, including the terminal state.

$\subset$ subset, e.g., $\mathcal{R} \subset \mathbb{R}$.

a an action.

$p(s', r|s, a)$ probability of transition to state s' with reward r, from state s and action a.

$p(s'|s, a)$ probability of transition to state s', from state s and action a.

$q_*(s, a)$ value taking action a in state s under the optimal policy.

$q_\pi(s, a)$ value taking action a in state s under policy π.

r a reward.

$r(s, a, s')$ expected immediate reward on transition from s to s' under action a.

s a state.

$v_*(s)$ value of state s under optimal policy.

$v_\pi(s)$ value of state s under the optimal policy π (expected return).

Greek symbols

π policy, decision-making rule.

$\pi\left(a|s,\ \boldsymbol{\theta}\right)$ probability of taking action a in state s given parameter $\boldsymbol{\theta}$.

$\pi\left(a|s\right)$ probablity of taking action a in state s under *stochastic* policy π.

$\pi\left(s\right)$ action taken in state s under *deterministic* policy π.

Contents

List of Figures vii

List of Tables ix

1 Introduction 1

2 Orbital ascent trajectory optimization 3
- 2.1 Analytical solution . . . 3
  - 2.1.1 Single stage performance . . . 3
  - 2.1.2 Optimization of multistage rockets . . . 5
  - 2.1.3 Optimal thrust attitude angle policy . . . 6
  - 2.1.4 Optimal propellant utilization policy . . . 8
  - 2.1.5 Gravity turn . . . 11
- 2.2 Trajectory Optimization . . . 12
  - 2.2.1 Closed and open loop optimal control solutions . . . 13
  - 2.2.2 Trajectory optimization problem . . . 13
  - 2.2.3 Transcription . . . 14

3 Reinforcement learning 17
- 3.1 Finite Markov Decision Process . . . 17
- 3.2 Elements of Reinforcement Learning . . . 17
- 3.3 Expected return . . . 18
- 3.4 Policy and value functions . . . 19
- 3.5 K-armed bandit problem . . . 19
- 3.6 ϵ–greedy policies . . . 19
- 3.7 Dynamic Programming . . . 20
  - 3.7.1 Policy evaluation . . . 21
  - 3.7.2 Policy improvement . . . 22
  - 3.7.3 Policy iteration . . . 23
  - 3.7.4 Generalized Policy Iteration . . . 23
- 3.8 Monte Carlo Methods . . . 23
- 3.9 Temporal Difference . . . 23
  - 3.9.1 TD Prediction . . . 24
  - 3.9.2 Sarsa: On-policy TD Control . . . 24
  - 3.9.3 Q-learning: Off-policy TD control . . . 24
  - 3.9.4 n-step Bootstrapping . . . 25
- 3.10 Approximate solutions . . . 26
  - 3.10.1 Stochastic Gradient Decent methods . . . 26
  - 3.10.2 Adam . . . 27
  - 3.10.3 Artificial Neural Networks . . . 28
  - 3.10.4 LSTM Neural Networks . . . 28
- 3.11 Policy Gradient Methods . . . 29
  - 3.11.1 Policy parameterization . . . 29
  - 3.11.2 Policy gradient theorem . . . 30
  - 3.11.3 REINFORCE . . . 30
  - 3.11.4 REINFORCE with baseline . . . 31
  - 3.11.5 Actor-Critic . . . 31
  - 3.11.6 PPO . . . 32

4 Conclusion 35

5 Proposed thesis research 37

Bibliography **39**

1 Introduction

Thanks to advancements in technology and an increasingly widespread use of satellite services, demand for space access has been steadily rising the past decades. This growth in the satellite launch services market has lead to the development of new launchers by both government agencies and an emerging private sector.

One often overlooked, but crucial step during the design process of any product is the performance evaluation of proposed designs. This is the process of determining a products ability to meet the design requirements. Too low of a performance, and the product will not be able to fullfil its purpose. To high of a performance and the product may be over engineered or inefficient.

In the case of a launcher the performance may be measured by how much payload mass it can bring to it's target range of orbits. In other words, in order to maximize the performance of a launcher, the propellant mass fraction of the upper stage needs to be reduced.

A successful orbital insertion requires the launcher to carefully follow an optimized trajectory. This trajectory has a massive effect on the maximum performance of the launcher, but is unfortunately not a trivial problem to solve. There has been quite some research in this trajectory optimization problem, but due to the complexity, the most reliable way to get the optimum is by using a global optimization method such as Genetic Algorithm (GA) or Participle Swarm Optimization (PSO).

Using a global optimizer to solve this problem is computationally intensive. This is not an issue when computing the trajectory for one particular launch during normal operations, but it is during the design process of the launch vehicle. A fast trajectory optimizer is desired during the design process in order to for example get a large number of results when performing a sensitivity analysis on the design parameters or continuously evaluate and monitor the feasibility of a changing design.

This thesis proposes the use of Reinforcement Learning (RL) as a possible solution to the trajectory optimization problem. With enough training an RL agent is able to learn to solve complex and non-linear problems. The goal is to sufficiently train an RL agent in order for it to be able to compute the optimal or close to optimal trajectory of a particular launcher design to a specific target orbit with just one simulation run.

Even if generating optimal trajectories using RL ends up being unfeasible, it may still generate close to optimal trajectories, which may either be acceptable for a design parameter sensitivity analysis, or serve as an initial guess for a more refined optimization algorithm.

The use of Reinforcement Learning (RL) in spacecraft guidance, navigation and control opens up the possibilities to solve future problems as space exploration becomes more prevalent. For example thanks to it's ability to run and adapt in real time, it may be used in situations that are difficult to forsee or too cumbersome to manually take into account, such as hovering and manoeuvering around the non-uniform, rotating gravitational field of an asteroid.

this citation, it's so latex doesn't

The objective of this thesis is to develop a orbital ascent trajectory optimization method using reinforcement learning and investigate its effectiveness compared to existing methods.

How are orbital ascent trajectories currently optimized?

- What are the challenges faced when optimizing an orbital ascent trajectory?
- What methods are currently used or being researched to solve this problem?

What is Reinforcement Learning?

- How does RL work?
- What is RL capable of?
- How has RL been used so far?
- What state of the art technological developments make use of RL?

How can Reinforcement Learning methods be used to solve the orbital ascent trajectory optimization problem?

- What advantages do RL methods have over existing methods?
- What elements of Reinforcement Learning can be used to optimize orbital ascent trajectories?
- How do these newly developed methods compare to existing methods?
- May these RL methods be used to compliment or improve the performance of existing methods?

2

Orbital ascent trajectory optimization

The problem of placing a satellite in orbit can be solved by achieving a required velocity at a certain position in space. Unfortunately reaching this goal is not a trivial problem. In order for a launcher to carry out it's mission as efficient as possible it must follow a carefully optimized trajectory. Failure to do so results in wasted potential performance.

Rewrite this once is done

This chapter starts with a analytical solution by simplifying the problem and continues with methods with which the full trajectory optimization problem may be solved.

2.1. Analytical solution

The analytical solution to the orbital ascent trajectory optimization problem as described in *Introduction to space dynamics by William T. Thomson* [11] is shown in this section.

2.1.1. Single stage performance

A rocket is a variable mass vehicle which produces thrust by ejecting high speed particles. Neglecting atmospheric forces and assuming a uniform gravitational field during vertical flight results in the following equation for the vehicle's acceleration $\frac{dv}{dt}$, where T is the thrust, m the mass and g_0 the gravitational acceleration at the earth's surface.

$$\frac{dv}{dt} = \frac{T}{m} - g_0 \tag{2.1}$$

Since the rocket is constantly losing mass, the mass flow $\frac{dm}{dt}$ is negative and thus the thrust can be written as shown below if a term due to the difference in exhaust and ambient pressure is neglected.

$$T = -u \cdot \frac{dm}{dt} \tag{2.2}$$

Here u is the velocity of the expelled mass, also known as the exhaust velocity. The specific impulse I_{sp} is a measure of the performance of a rocket engine and it's relationship to the exhaust velocity as taken from the book [11] is as follows,

$$u = g_0 I_{sp} \tag{2.3}$$

Substituting Equation (2.2) into Equation (2.1) and integrating yields the following equation for the velocity v.

$$v - v_0 = u \ln \frac{m_0}{m} - g_0 t \tag{2.4}$$

Where m is the mass at any time t. Substituting the burnout time t_{b0} and burnout mass m_{b0} into the equation results in an expression for the maximum attainable velocity for vertical flight.

$$v_{b0} - v_0 = u \ln \frac{m_0}{m_{b0}} - g_0 t_{b0} \tag{2.5}$$

The thrust parameter $\mathcal{R}$ is now introduced. This is the initial thrust over mass ratio and its relationship to the initial acceleration a_0 is as follows,

$$\mathcal{R} = \frac{T}{m_0} = \frac{a_0}{g_0} + 1 \tag{2.6}$$

The duration for powered flight is then,

$$t_{b0} = g_0 I_{sp} \left(\frac{m_0 - m_{b0}}{T} \right) = \frac{I_{sp}}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) \tag{2.7}$$

Using Equations (2.3) and (2.7), Equation (2.5) can now be written as,

$$v_{b0} - v_0 = g_0 I_{sp} \left[\ln \frac{m_0}{m_{b0}} - \frac{1}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) \right] \tag{2.8}$$

It is possible to determine the distance traveled during powered flight by integrating Equation (2.4). A realistic assumption is that the mass during flight is known and that a constant rate of fuel mass leads to constant thrust. Thus the following can be defined,

$$dt = \frac{dm}{\dot{m}} \qquad \text{and} \qquad -\dot{m} = \frac{m_0 - m_{b0}}{t_{b0}} = \text{constant} \tag{2.9}$$

so that

$$\int_0^{t_{b0}} (\ln m)\, dt = \frac{1}{\dot{m}} \int_{m_0}^{m_{b0}} (\ln m)\, dm = \frac{m}{\dot{m}} (\ln m - 1) \Big|_{m_0}^{m_{b0}} \tag{2.10}$$

$$= \frac{m_0 - m_{b0}}{\dot{m}} - \frac{m_0}{\dot{m}} \ln m_0 + \frac{m_{b0}}{\dot{m}} \ln m_{b0} \tag{2.11}$$

The distance traveled then becomes,

$$h_{b0} = u t_{b0} \left[1 - \frac{1}{(m_0/m_{b0} - 1)} \ln \frac{m_0}{m_{b0}} \right] + v_0 t_{b0} - \frac{1}{2} g_0 t_{b0}^2 \tag{2.12}$$

$$= g_0 \frac{I_{sp}^2}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) \left[1 - \frac{1}{(m_0/m_{b0} - 1)} \ln \frac{m_0}{m_{b0}} \right] + v_0 \frac{I_{sp}}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) - \frac{1}{2} g_0 \frac{I_{sp}^2}{\mathcal{R}^2} \left(1 - \frac{m_{b0}}{m_0} \right)^2 \tag{2.13}$$

After burnout the rocket continues coasting with the velocity only changing due to the gravitational acceleration. Unfortunately due to the altitude in this phase of flight it is not possible to assume a uniform gravitational field.

Since the system does not lose or gain energy while coasting it is possible to get the distance traveled by equating the kinetic energy at burnout with the work done by the *gravitational force* $m_{b0} g_0 \left(\frac{R}{r} \right)^2$. Thus the equation for coasting distance becomes,

$$h_c = \frac{v_{b0^2}}{2g_0} \frac{(R + h_{b0})^2}{R^2 - (v_{b0}^2/2g_0)(R + h_{b0})} \tag{2.14}$$

where $r_{b0} = R + h_{b0}$ has been substituted. The total height can then be calculated by summing Equations (2.12) and (2.14).

Equations (2.8) and (2.14) indicate that the performance of a single stage rocket depends on the specific impulse I_{sp} and thrust ratio $\mathcal{R}$ and the mass ratio $\mu = \frac{m_0}{m_{b0}}$. The effect of varying these quantities on the burnout velocity or height can be found by considering Equations (2.8) and (2.12) to be in the form,

$$v_{b0} = f_1\left(I_{sp}, \mathcal{R}, \mu\right) \tag{2.15}$$

$$h_{b0} = f_2\left(I_{sp}, \mathcal{R}, \mu\right) \tag{2.16}$$

The change in burnout velocity is then defined by the following equation,

$$dv_{b0} = \frac{\delta f_1}{\delta I_{sp}} dI_{sp} + \frac{\delta f_1}{\delta \mathcal{R}} d\mathcal{R}_{sp} + \frac{\delta f_1}{\delta \mu} d\mu \tag{2.17}$$

The optimum burnout velocity will be at $dv_{b0} = 0$, this defines the constraint on the three quantities.

2.1.2. Optimization of multistage rockets

It can be shown using the equations from the previous section that with achievable mass ratios, thrust ratios and specific impulse, it is not possible to reach earth orbital velocities using single stage rockets. Thus multistage launchers are required for space travel.

In a multistage rocket the burnout velocity of the previous stage is the initial velocity of the next. The maximum velocity by multistage rockets can then be computed as the sum of the single stage velocities given by Equation (2.5). Ignoring gravity loss due to burn-time results in the following equation for the maximum velocity of an N-stage rocket,

$$v_m = \sum_{i=1}^{N} u_i \ln \mu_i \tag{2.18}$$

Assuming this velocity v_m is specified, the mass ratios for each stage must be chosen such that the overall mass ratio $\frac{m_{01}}{P}$ is minimized, where m_{01} is the take-off mass and P the payload mass. To do this it is necessary to express the overall mass ratio in terms of the individual mass ratios μ_i.

Since the empty structure of the stages are to be discarded, the initial mass of each new stage is equal to the initial mass of the previous stage minus the propellant burned and the empty structure that is thrown off. Thus the overall mass ratio may be written as follows.

$$\frac{m_{01}}{P} = \frac{m_{01}}{m_{01} - m_{p1} - m_{s1}} \frac{m_{02}}{m_{02} - m_{p2} - m_{s2}} \dots \frac{m_{0N}}{P} \tag{2.19}$$

The structural factor β_i is now introduced.

$$\beta_i = \frac{m_{si}}{m_{pi} + m_{si}} \tag{2.20}$$

Equation (2.19) can then be rewritten into,

$$\frac{m_{01}}{P} = \frac{\mu_1 (1 - \beta_1)}{1 - \mu_1 \beta_1} \frac{\mu_2 (1 - \beta_2)}{1 - \mu_2 \beta_2} \dots \frac{\mu_N (1 - \beta_N)}{1 - \mu_N \beta_N} \tag{2.21}$$

If $\frac{m_{01}}{P}$ needs to be minimized, then the same result can be achieved my minimizing $\ln \frac{m01}{P}$ as well.

$$\ln \frac{m_{01}}{P} = \sum_{i=1}^{N} \ln \frac{\mu_i (1-\beta_i)}{1-\mu_i \beta_i} = \sum_{i=1}^{N} [\ln \mu_i + \ln (1-\beta_i) - \ln (1-\mu_i \beta_i)] \tag{2.22}$$

This equation does not contain the constraint imposed by the specified velocity in Equation 2.18. It can be added using the Lagrange multiplier method which results in the following,

$$\ln \frac{m_{01}}{P} = \sum_{i=1}^{N} [\ln \mu_i + \ln (1-\beta_i) - \ln (1-\mu_i \beta_i) + \lambda \{u_i \ln \mu_i - v_m\}] \tag{2.23}$$

In order to solve for the optimal mass ratios μ_i, Equation (2.23) must be differentiated with respect to μ_i. Performing this differentiation and rewriting results in N equations in the form of,

$$\mu_i = \frac{1+\lambda u_i}{\lambda u_i \beta_i} \tag{2.24}$$

Substituting this into Equation (2.18) yields the following expression which, assuming u_i, β_i and v_m are known, may be used to solve for λ.

$$v_m = \sum_{i=1}^{N} u_i \ln \frac{1+\lambda u_i}{\lambda u_i \beta_i} \tag{2.25}$$

With λ known the mass ratios μ_i may be calculated using Equation (2.24).

2.1.3. Optimal thrust attitude angle policy

To simplify this problem the rocket is assumed to be rigid and the aerodynamics are neglected. In general the length of the powered flight path is short compared to the earth's radius, thus it's safe to replace the central gravitational field by a constant-plane parallel-force field.

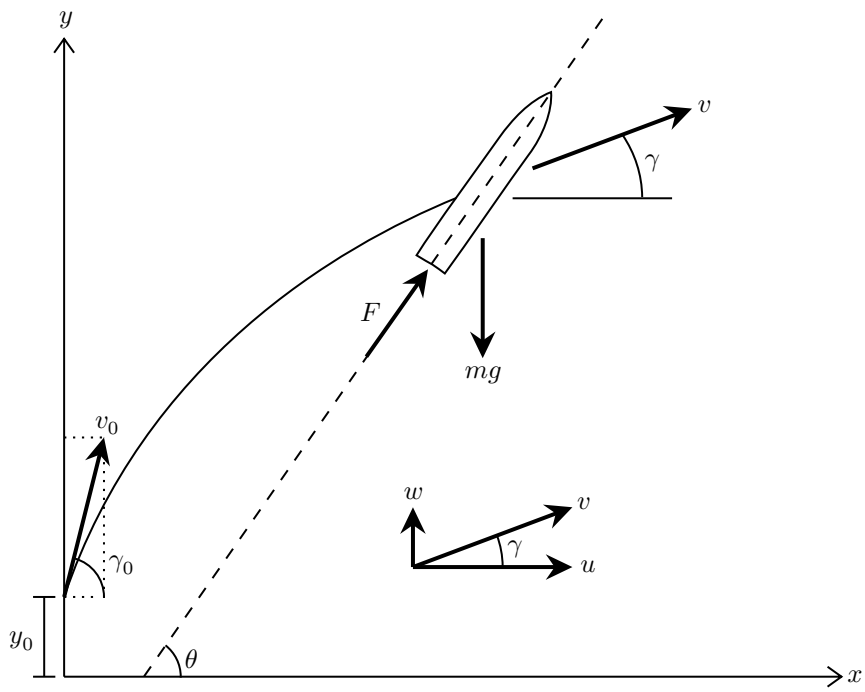


Figure 2.1: Powered flight trajectory[11].

The geometry of the problem is shown in Figure 2.1. Where θ is the thrust attitude angle and γ the flight path angle. Gravity acts along to the negative y axis. Letting u and w be the x and y component of the velocity v results in the following equations of motion.

$$\dot{u} = \frac{F}{m}\cos\theta \tag{2.26}$$

$$\dot{w} = \frac{F}{m}\sin\theta - g \tag{2.27}$$

$$\dot{y} = w \tag{2.28}$$

$\frac{F}{m}$ and θ vary with time and have a significant effect on the trajectory. Assuming that $\frac{F}{m}$ is known then the problem can be defined as finding the $\theta(t)$ that results the maximum horizontal velocity U at a specified altitude Y. The time T corresponds to the instant in time where $y = Y$. Since the timing of this event depends on $\theta(t)$, T is unknown. However it is assumed that all propellant is burned before $t = T$.

The problem can thus be defined as maximizing the time integral of Equation (2.26) shown below, with constraints.

$$U = u_0 + \int_0^T \frac{F}{m}\cos\theta\, dt \tag{2.29}$$

The goal is to reach this maximum at a specified value of y under the condition that $w_{t=T} = 0$. So Equations (2.27) and (2.28) are the constraints that limit the allowable variations of Equation (2.29). Using the Lagrange multiplier method to add these constraints results in the following equation.

$$U = u_0 + \int_0^T \left[\frac{F}{m}\cos\theta + \lambda_1\left(\dot{w} - \frac{F}{m}\sin\theta + g\right) + \lambda_2\left(\dot{y} - w\right)\right] dt \tag{2.30}$$

With the boundary conditions being,

At $t = 0$

$$y = y_0 \qquad x = 0$$
$$u_0 = v_0\cos\gamma_0 \qquad w_0 = w_0\sin\gamma_0$$

At $t = T$

$$y = Y \qquad w = 0$$

The initial velocity v_0 is considered fixed, but the initial flight path angle γ_0 is left undetermined.

This optimization is solved using variational calculus. Thus the next step is to get an equation for δU, the variation of Equation (2.30). However determining the variation of an integral is not trivial. According to derivation in appendix C of the book [11], the variation of the following integral is shown in Equation (2.32).

$$I = \int_a^b f\left(t, z, \frac{dz}{dt}\right) \tag{2.31}$$

$$\delta I = \frac{\delta f}{\delta z'}\delta z\Big|_a^b + \int_a^b \left(\frac{\delta f}{\delta z} - \frac{d}{dt}\frac{\delta f}{\delta z'}\right)\delta z\, dt \tag{2.32}$$

Applying Equation (2.32) to Equation (2.30) results in the following equation for the variation of U.

$$\begin{aligned}\delta U = &- v_0\sin\gamma_0\,\delta\gamma_0 + \lambda_1\delta w\Big|_0^T + \lambda_2\delta y\Big|_0^T \\ &- \int_0^T \left[\frac{F}{m}\left(\sin\theta + \lambda_1\cos\theta\right)\delta\theta + \frac{d\lambda_2}{dt}\delta y + \left(\frac{d\lambda_1}{dt} + \lambda_2\right)\delta w\right] dt = 0\end{aligned} \tag{2.33}$$

Since the value of y is known at $t = 0$ and $t = T$, it's variation at the endpoints must be zero, thus the third term can be eliminated. The variation δw at $t = 0$ is,

$$\delta w\Big|_0 = \delta(v_0 \sin\gamma_0) = v_0 \cos\gamma_0 \; \delta\gamma_0 \tag{2.34}$$

At $t = T$, w can be calculated by integrating Equation (2.27) into the equation shown below.

$$w = \int_0^T \frac{F}{m} \sin\theta dt - gT + v_0 \sin\gamma_0 \tag{2.35}$$

Thus it's variation at $t = T$ is,

$$\delta w\Big|^T = \int_0^T \frac{F}{m} \cos\theta \; \delta\theta \; dt + v_0 \; \delta\gamma_0 \sin\gamma_0 \tag{2.36}$$

For the optimal curve w becomes zero at $t = T$, however for the varied curve it becomes zero at a different time $t = T + \delta T$. Since $\frac{F}{m}$ is not varied for $t > T$ the expression for $\delta w|^T$ must equal $g\delta T$. Substituting these equations into Equation (2.33) yields,

$$\begin{aligned} \delta U = &- v_0(\sin\gamma_0 + \lambda_{1,t=0}\cos\gamma_0)\; \delta\gamma_0 + g\lambda_{1,t=T}\; \delta T \\ &- \int_0^T \left[\frac{F}{m}\left(\sin\theta + \lambda_1\cos\theta\right)\delta\theta + \frac{d\lambda_2}{dt}\delta y + \left(\frac{d\lambda_1}{dt} + \lambda_2\right)\delta w\right] dt = 0 \end{aligned} \tag{2.37}$$

Since the rest of the variations are arbitrary, δU can only be zero if

$$\tan\gamma_0 = -\lambda_{1,t=0} \tag{2.38}$$

$$\lambda_{1,t=T} = 0 \tag{2.39}$$

$$\tan\theta = -\lambda_1 \tag{2.40}$$

$$\frac{d\lambda_2}{dt} = 0 \tag{2.41}$$

$$\frac{d\lambda_1}{dt} + \lambda_2 = 0 \tag{2.42}$$

The last two equations give,

$$\lambda_2 = -C_2 \qquad \text{a constant} \tag{2.43}$$

$$\frac{d\lambda_1}{dt} - C_2 = 0 \tag{2.44}$$

$$\lambda_1 = C_1 + C_2 t \tag{2.45}$$

Substituting λ_1 for $t = 0$ and $t = T$ into Equations (2.38) and (2.39), results in expressions for constants C_1 and C_2.

$$C_1 = -\tan\gamma_0 \tag{2.46}$$

$$C_2 = \frac{1}{T}\tan\gamma_0 \tag{2.47}$$

Combining Equations (2.40, 2.45, 2.46, 2.47) yields an expression for the thrust attitude angle θ.

$$\tan\theta = \left(1 - \frac{t}{T}\right)\tan\gamma_0 \tag{2.48}$$

Equation (2.48) gives an expression for the thrust attitude angle θ if the initial flight path angle γ_0 is allowed to variate. If γ_0 is given a fixed quantity, a similar procedure starting from Equation (2.33) can be followed with the added condition that $\delta\gamma_0 = 0$. This results in the following.

$$\tan\theta = \tan\theta_0 - C_2 t \tag{2.49}$$

2.1.4. Optimal propellant utilization policy

The previous section resulted in the optimal thrust attitude angle θ policy that results in a maximum horizontal velocity U for a specified thrust ratio $\frac{F}{m}$ policy. However the maximum attainable U still depends on what policy is chosen for $\frac{F}{m}$ or in other words, the manner of propellant utilization.

It is important to note that there is a maximum attainable velocity V_k. From Equation (2.5) it is clear that the burn-time has a negative effect on V_k, thus the optimal function for $V(t)$ must be discontinues, in other words, the propellant must be burned instantly. By neglecting all forces except thrust, V_k is only a function of the specific impulse and mass ratio.

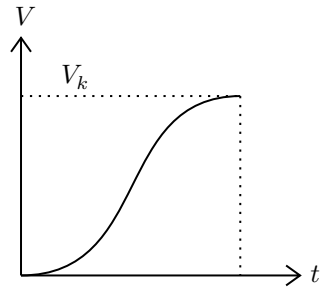


Figure 2.2: Possible variation of V

$$V_k = gI_{sp} \ln \frac{m_0}{m_{b0}} \tag{2.50}$$

Assuming the specific impulse is independent of the propellant utilization policy, it is convenient to make the following substitutions.

$$\frac{dV}{dt} = \frac{F}{m} \tag{2.51}$$

$$V = \int_0^t \frac{F}{m} dt \tag{2.52}$$

Here is V the rocket thrust velocity with the maximum attainable value being V_k. V is only a function of time and depends on the manner in which fuel is consumed. Both F and m are positive values thus the slope for V is bounded between 0 and infinity. Figure 2.2 shows a possible variation of V.

Assuming the initial values v_0 and γ_0 are fixed and that the policy for the thrust attitude angle in Equation (2.49) is followed. Then Equation (2.30) may be rewritten into,

$$U = \int_0^T \left[(\cos\theta - \lambda_1 \sin\theta) \frac{dV}{dt} + \lambda_1 (\dot{w} + g) + \lambda_2 (\dot{y} - w) \right] dt \tag{2.53}$$

Since the optimal thrust attitude policy is followed, then all variations have already been handled in the previous section, except for δV. Thus only the first term of the integral needs to be considered.

In order to apply Equation (2.32) to Equation (2.53) the following is needed,

$$\frac{\delta f}{\delta V} = 0 \qquad \frac{\delta f}{\delta \dot{V}} = (\cos\theta - \lambda_1 \sin\theta) \tag{2.54}$$

Since the values for V at the start and end time are fixed, it's variation δV at $t = 0$ and $t = T$ are zero. Thus the variation of Equation (2.53) is as follows,

$$\begin{aligned} \delta U &= -\int_0^t \frac{d}{dt} (\cos\theta - \lambda_1 \sin\theta) \delta V dt \\ &= \int_0^t \left[(\sin\theta + \lambda_1 \cos\theta) \dot{\theta} + \sin\theta \frac{d\lambda_1}{dt} \right] \delta V dt \end{aligned} \tag{2.55}$$

From Equations (2.55) and (2.49) it's possible to get,

$$\lambda_1 = -\tan\theta = -\tan\theta_0 + C_2 t \tag{2.56}$$

$$\frac{\lambda_1}{dt} = -\frac{\dot{\theta}}{\cos^2\theta} = C_2 \tag{2.57}$$

Using these expressions, δU may be written as,

$$\delta U = C_2 \int_0^T (\sin\theta)\, \delta V dt \tag{2.58}$$

$$= C_2 \int_0^T \frac{\tan\theta_0 - C_2 t}{\sqrt{1 + (\tan\theta_0 - C_2 t)^2}} \delta V dt \tag{2.59}$$

$$= C_2 \int_0^T \Phi\ \delta V dt = 0 \tag{2.60}$$

Where,

$$\Phi = \frac{\tan\theta_0 - C_2 t}{\sqrt{1 + (\tan\theta_0 - C_2 t)^2}} \tag{2.61}$$

Before continuing some details are reiterated.

- Since U is being maximized, any variation δU must be negative.
- The velocity V may not exceed V_k.
- Since the slope of V is bounded between zero and infinity, it is not possible for V to be negative.
- The optimal function for $V(t)$ is discreet, meaning only instantaneous burns.

It is possible for Φ to be always positive, always negative or vary from positive to negative with time.

1. If Φ is always positive, then any variation δV must be negative in order for δU to be negative. Since $V(t)$ must always be larger than zero, then all the propellant must be burned instantaneously at $t = 0$ so that any variation δV from the optimal $V(t)$ is negative.
2. If Φ is always negative, then any variation δV must be positive in order for δU to be negative. Since $V(t)$ must always be lower than V_k, then all the propellant must be burned instantaneously at $t = T$.
3. If Φ varies from positive to negative. Then part of the propellant must be instantaneously burned at $t = 0$, and the rest and $t = T$.

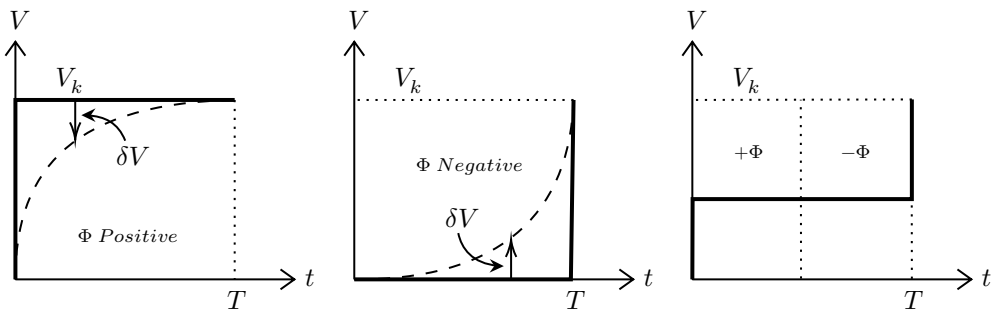


Figure 2.3: Optimal $V(t)$ for different options for Φ[11].

2.1.5. Gravity turn

It was previously shown that the optimal thrust attitude angle policy is given by the following equation,

$$\tan\theta = \left(1 - \frac{t}{T}\right)\tan\gamma_0 \tag{2.62}$$

One drawback of this policy is that it results in large angles of attack $\alpha = \theta - \gamma$. This is not an issue when flying outside of the atmosphere, but when flying within the atmosphere it results in increased aerodynamic forces, thus increasing aerodynamic losses and risk of structural damage. Thus for flights within the atmosphere a trajectory known as the *gravity turn* is followed. In a gravity turn the thrust vector is kept parallel to the velocity vector at all times starting from a non-vertical initial velocity v_0. This is a zero angle of attack or zero lift trajectory.

For convenience the flight path angle ψ is now measured from the vertical. By neglecting aerodynamic forces and assuming a uniform gravitational field, it is possible to write the equation for the forces along the tangential and normal directions as follows,

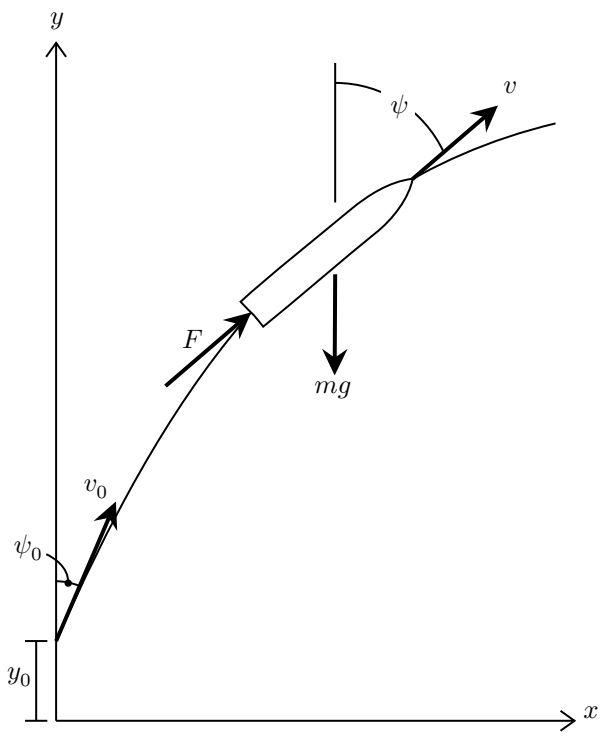


Figure 2.4: Gravity turn[11].

$$\frac{1}{g}\frac{dv}{dg} = \frac{F}{mg} - \cos\psi \tag{2.63}$$

$$\frac{v}{g}\frac{d\psi}{dt} = \sin\psi \tag{2.64}$$

These equations are non-linear and there is no analytical solutions when $\frac{F}{mg}$ varies, thus $\frac{F}{mg}$ needs to be constant in order to solve the gravity turn trajectory. Since $m = m_0 - \dot{m}t$, this means that the thrust F must decrease with time as well.

However in reality it is reasonable to assume that $F = constant$. Thus for a realistic solutions it's not possible to assume that $\frac{F}{mg}$ doesn't vary. One way solve it, is by assuming $\frac{F}{mg}$ is constant for a short amount of time and perform a step-by-step numerical integration using the analytical solution for constant $\frac{F}{mg}$ on each interval.

A new parameter z is introduced and defined as,

$$z = \tan\frac{1}{2}\psi = \sqrt{\frac{1-\cos\psi}{1+\cos\psi}} = \frac{\sin\psi}{1+\cos\psi} \tag{2.65}$$

Then

$$z^2 = \frac{1-\cos\psi}{1+\cos\psi} \tag{2.66}$$

and

$$\frac{1-z^2}{1+z^2} = \cos\psi \tag{2.67}$$

Differentiating Equation (2.65),

$$\frac{dz}{dt} = \frac{1}{2\cos^2\left(\frac{\psi}{2}\right)}\frac{d\psi}{dt} \tag{2.68}$$

Thus,

$$\frac{d\psi}{dt} = (1 + \cos\psi)\frac{dz}{dt} \tag{2.69}$$

By defining $n = \frac{F}{mg}$ and substituting the previous results into Equations (2.63) and (2.64) results in,

$$\frac{1}{g}\frac{dv}{dt} = n - \frac{1 - z^2}{1 + z^2} \tag{2.70}$$

$$\frac{v}{g}\frac{dz}{dt} = z \tag{2.71}$$

Eliminating dt between the two equations yields,

$$\frac{dv}{v} = n\frac{dz}{z} - \frac{1 - z^2}{1 + z^2}\frac{dz}{z} \tag{2.72}$$

Integrating,

$$\ln v = \ln z^n = \ln\frac{1 + z^2}{z} + \ln C' \tag{2.73}$$

or

$$v = Cz^{n-1}\left(1 + z^2\right) \tag{2.74}$$

The constant C can be calculated from the initial values with,

$$C = \frac{v_0}{z_0^{n-1}\left(1 + z_0^2\right)} \tag{2.75}$$

Substituting Equation (2.74) into (2.71) and integrating,

$$\begin{aligned} t &= \frac{C}{g}\int_{z_0}^{z} z^{n-2}\left(1 + z^2\right)dz \\ &= \frac{C}{g}z^{n-2}\left(\frac{1}{n-1} + \frac{z^2}{n+1}\right)\Bigg|_{z_0}^{z} \end{aligned} \tag{2.76}$$

Equation (2.76) gives t as a function of z and any initial condition C. Thus ψ is also known for any C and t. Equation (2.74) gives v in terms of C, t and z. So Equations (2.65), (2.75) and (2.76) give the analytical solution for the gravity turn with a constant $\frac{F}{mg}$.

In order to solve the gravity turn with varying $\frac{F}{mg}$ Equation (2.65) can be used to calculate values for z, Equation (2.75) to calculate C, define $n = \frac{F}{mg}$ at $t = 0$ and v can be calculated using Equation (2.74). Then a value of ψ slightly higher than ψ_0 must chosen and Δt can then be calculated from Equation (2.76). The increment in displacement is then,

$$\Delta x = \frac{1}{2}\left(v_0\sin\psi_0 + v\sin\psi\right)\Delta t \tag{2.77}$$

$$\Delta y = \frac{1}{2}\left(v_0\cos\psi_0 + v\cos\psi\right)\Delta t \tag{2.78}$$

This procedure can then be repeated with the values at the new point to obtain the full trajectory.

2.2. Trajectory Optimization

Unfortunately due to the complex non-linear dynamics and constraints, and high dimensionality of the problem it is not possible to calculate the optimal orbital ascent trajectory analytically and thus numerical solutions are necessary. This has been a problem since the beginning of space exploration and thus countless years of research have resulted in the use and further development of *trajectory optimization*, a set of mathematical techniques used to find the ideal or desired behavior of a dynamical system by finding the inputs, referred to as *controls*, to the system as functions of time.

2.2.1. Closed and open loop optimal control solutions

In itself trajectory optimization is class of methods used to compute solutions for optimal control problems. Within optimal control there are two types of solutions, closed-loop and open-loop.

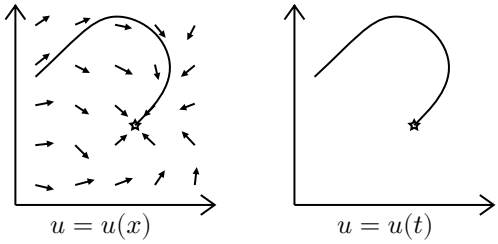


Figure 2.5: Optimal control closed and open loop solutions.

Closed-loop solutions provide the optimal controls for the entire state space, $u(x)$, thus advantages of these solutions include the possibility to compute the global optimal solution and the ability to apply the solution directly into real systems. The drawback is that these solutions are difficult to compute, specially when applied to higher dimensional problems. Dynamic Programming and the main topic of thesis, Reinforcement Learning work by finding an optimal policy. This is a deterministic function or probability distribution mapping states to actions and thus provide closed-loop solutions to optimal control problems [5]. Chapter 3 continues further with these methods.

On the other hand, trajectory optimization computes open-loop solutions consistent of an initial state and a sequence of controls defined as a function of time, $u(t)$. Together these define one single trajectory the system takes through the state space. The advantage of these solutions is that they are easier to compute, and thus more practical for higher dimensional problems. The disadvantage is that these solutions do not necessarily converge to the global optimal and require a stabilizing controller in order to keep the system on the predefined trajectory when applied to real systems [5].

2.2.2. Trajectory optimization problem

There are many ways to define trajectory optimization problems [7], but assuming the problem consists of a single-phase and is continues in time, the problem can be described with an objective function and a set of constraints as shown is Equation 2.79 [5].

$$
\begin{aligned}
\underset{t_0, t_F, x(t), u(t)}{\text{minimize}} \quad & J\left(t_0, t_F, x(t_0), x(t_F)\right) + \int_{t_0}^{t_F} w(\tau, x(\tau), u(\tau)) d\tau \\
\text{subject to} \quad & \dot{x}(t) = f(t, x(t), u(t)) \qquad \text{(system dynamics)} \\
& h(t, x(t), u(t)) \leq 0 \qquad \text{(non-linear path constraints)} \\
& g(t_0, t_F, x(t_0), x(t_F)) \leq 0 \qquad \text{(non-linear boundary constraints)} \\
& x_{low} \leq x(t) \leq x_{upp} \qquad \text{(path bounds on state)} \\
& u_{low} \leq u(t) \leq u_{upp} \qquad \text{(path bounds on control)} \\
& t_{low} \leq t_0 < t_F \leq t_{upp} \qquad \text{(bounds on initial and final time)} \\
& x_{0,low} \leq x(t_0) \leq x_{0,upp} \qquad \text{(bounds on initial state)} \\
& x_{F,low} \leq x(t_F) \leq x_{F,upp} \qquad \text{(bounds on final state)}
\end{aligned}
\tag{2.79}
$$

The objective function consists of two terms, a boundary objective, $J(\cdot)$, also known as the *Mayer term*, and a path integral along the entire trajectory with integrant, $w(\cdot)$, also known as the *Lagrange term*. Such an objective function is said to be in *Bolza form*.

The system dynamics constraint is typically non-linear and describes how the system changes with time. The non-linear path constraints enforce restrictions along the path, for example airspace restrictions. Non-linear boundary constraints restrict the initial and final state of the system. The path bounds put restrictions on the states and controls, for example maximum actuator settings or aerodynamic

loads. The bounds on initial and final state and time can be used to for example, set the initial launch location and final position and velocity needed for the target orbit.

2.2.3. Transcription

The goal of trajectory optimization is to minimize an objective function in the form of $J(f(t))$, where the decision variable $f(t)$ is an arbitrary vector function. In contrast, *parameter optimization* minimizes objective functions in the form of $J(x)$, where the decision variables in vector x are a finite set of real numbers. This makes trajectory optimization a more difficult problem since having a function as decision variable is the equivalent of having an infinite set of real numbered decision variables [5].

Transcription is the process of converting a trajectory optimization problem into a constrained parameter optimization problem. Thus the goal of transcription is to convert a infinite-dimensional problem into a finite-dimensional approximation, which can then be solved using existing numerical methods [2]. The three fundamental steps of the transcription method are:

1. Convert the dynamic system into a problem with a finite set of variables.
2. Solve the finite-dimensional problem using a parameter optimization method.
3. Asses the accuracy of the finite-dimensional problem approximation and if necessary repeat the transcription and optimization steps.

Direct and indirect methods

Transcription methods can be split into two categories, direct and indirect methods.

Indirect methods work by analytically deriving the necessary conditions for optimal control as found in *J. Betts* [2], discretizing them and solving them numerically. In essence, indirect methods find the minimum of the objective functions by computing the derivative, search for stationary points using root finding methods and then selecting the minimum using the second derivative [2, 5].

In contrast direct methods work by constructing a sequence of points, $x_0, x_1, ..., x^*$, such that the objective function is minimized with each subsequent point. Unlike indirect methods, direct methods only need to compare values for the objective function.

The advantage of indirect methods is the higher accuracy and a better measure for the accuracy of the solution. However these methods require prior knowledge of the system and even if this knowledge is available the required analytical derivations may be too difficult or not possible.

Furthermore indirect methods are not robust. The region of convergence tends to be smaller, thus better initialization is required. Initializing indirect methods require the initialization of the adjoint variables, which since they have no physical representation, are more difficult to initialize [2, 5].

Direct shooting

In direct single shooting methods the trajectory is approximated using a simulation. The decision variables in the parameter optimization problem are the initial state and an open-loop parametrization of the controls along the trajectory.

An extension to direct single shooting methods are direct multiple shooting methods. Instead of representing the entire trajectory as a single simulation, the trajectory is segmented and each segment is represented by a simulation. Multiple shooting tends to be more robust than single shooting [5].

Direct collocation

Direct collocation methods work by approximating all of the continues functions as polynomial splines. As an example take the following problem, compute the trajectory to move a block between two points. Where x is the position, v the velocity and u the control. The objective function minimizes the control effort by integrating the square the control.

$$\begin{aligned} \underset{x(t),v(t),u(t)}{\text{minimize}} \quad & \int_0^1 u^2(\tau)\, d\tau \\ \text{subject to} \quad & \dot{x}(t) = v(t) \\ & \dot{v}(t) = u(t) \\ & x(0) = 0 \\ & x(1) = 1 \\ & v(0) = 0 \\ & v(1) = 0 \end{aligned} \tag{2.80}$$

The first step is to discretize the trajectory by representing the continues functions by their values at specific points in time, these are known as *collocation points*.

$$\begin{aligned} t &\rightarrow t_0 \dots t_k \dots t_N \\ x &\rightarrow x_0 \dots x_k \dots x_N \\ v &\rightarrow v_0 \dots v_k \dots v_N \end{aligned}$$

The next step is to convert the continuos system dynamics to a set of constraints. This is done by equating the change in states between two collocation points to an estimate of the system dynamics integral during the same time period. Here trapezoidal collocation is used, where the system dynamics is approximated by piece-wise linear functions. Another alternative is the Hermite–Simpson collocation method, which approximates the system dynamics using piece-wise quadratic functions.

$$x_{k+1} - x_k = \frac{1}{2}(t_{k+1} - t_k)(v_{k+1} + v_k) \tag{2.81}$$

$$v_{k+1} - v_k = \frac{1}{2}(t_{k+1} - t_k)(u_{k+1} + u_k) \tag{2.82}$$

$$\tag{2.83}$$

These constraints are known as *collocation constraints* and are applied to each pair of collocation points. Finally the objective function can then be approximated using quadrature by converting it to a summation of piece-wise functions between each collocation point. Using trapezoidal quadrature yields,

$$\underset{u(t)}{\text{minimize}} \int_0^1 u^2(\tau)\, d\tau \quad \approx \quad \underset{u_0 \dots u_N}{\text{minimize}} \sum_{k=0}^{N-1} \frac{1}{2}(t_{k+1} - t_k)(u_{k+1}^2 + u_k^2) \tag{2.84}$$

As result all continues functions have been replaced by constraints and the decision variables $x(t)$, $v(t)$ and $u(t)$ have been replaced by the finite sets of decision variables, $x_0 \dots x_N$, $v_0 \dots v_N$ and $u_0 \dots u_N$.

3

Reinforcement learning

Reinforcement Learning (RL) is a method of machine learning different from other machine learning methods such as supervised learning which uses a set of labeled examples from a knowledgeable supervisor, and unsupervised learning which is typically used to find hidden patterns in unlabeled data. In RL the learner is not told what action to take to achieve its goal, but instead it maps situations and actions by trying them out and discovering the most valuable. It does this by not only looking at the reward it received by its latest action, but it also takes into account any future rewards it could receive in the future by taking that action.

3.1. Finite Markov Decision Process

A Markov Decision Process (MDP) is a stochastic process that provides a mathematical framework used to describe problems where a learner learns from its actions to achieve an end goal. This framework is used in many areas, such as robotics, economics, manufacturing and in the case of this thesis, Reinforcement Learning.

The two main components of a MDP are the agent and the environment. The agent makes decisions, interacts with its environment and learns. The environment consists of everything else. The two are constantly interacting with each other, the agent will perform an action which changes the state of the environment and the environment responds by giving it a reward. The goal of the agent is to maximize the reward it receives from the environment.

This happens in a sequence of discrete time steps. At the beginning of each time step the environment is at a certain state s. The agent then decides based on its policy π which action a to take, this in turn changes the environment to the new state s' and the environment then gives the agent a certain reward r. In order for it to be a Markov Decision Process (MDP) this transition and the reward must satisfy the Marcov property, which means it needs to be memoryless and thus the environment's behavior must be independent of its past states. In other words, the probability of the environment to go from state s to s' and the reward r it gives back to the agent, must only be dependent on the initial state s and action a.

3.2. Elements of Reinforcement Learning

The main goal when solving an MDP problem is to find a policy π that defines the action $\pi(s)$ the agent will take in order to maximize the sum of rewards it receives. It characterizes the way in which the agent will behave at a given time by mapping the agents perceived state of the environment to the actions taken in those states. This policy may be as simple as a table or consist of several Neural Networks. The policy is the core of the agent in that it alone defines the behavior of the agent.

On each time step the environment will send the agent a reward. The goal of the agent is to maximize this reward in the long run. Thus this reward is used to tell the agent which states are good or bad. If the agent receives a low reward for a certain action it can change the policy so it takes a different

action in the future.

The reward signal tells the agent what the immediate effects of its action are. However, its goal is to maximize the cumulative sum of rewards it receives over time, and thus in order for it to receive higher rewards in the future, it might be necessary for the agent to take actions that yield lower rewards with the promise that it will receive higher rewards later. So the value is an indication of the reward the agent can expect to receive in the future by taking a certain action, and thus it is the main parameter used in the decision making process.

By defining the state and action taken during step t as S_t and A_t respectively, and its corresponding reward as R_t we can define the true value of an arbitrary action a as:

$$q_*(s,a) \doteq \mathbb{E}\left[R_t \mid S_t = s, A_t = a\right] \tag{3.1}$$

If $q_*(s,a)$ was known, then the solution would be trivial. Just select the action with the highest value. So it's assumed that the action values are not known, but are instead estimated from the rewards the agent receives. The estimated value of the state-action pair (s,a) at time t is known as $Q_t(s,a)$, thus $Q_t(s,a)$ needs to be as close to $q_*(s,a)$ as possible.

Finally some RL agents make use of a model for planning. This allows these agents to choose actions based on possible future situations before actually being experienced.

3.3. Expected return

The goal of the agent is to maximize the reward it receives over time. In order to formalize this goal, the *expected return* G_t can be defined as a function of the sequence of rewards. In it's simplest form this can be the cumulative sum of all rewards after time step t until the final time step T.

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + ... + R_T \tag{3.2}$$

This function assumes that there are one or more terminal states where the agent-environment interaction ends and break naturally into subsequences called *episodes*. A good example are plays of a game or any type of repeating task. Even if there are different terminal states, eg. winning or losing a game, the starting state of the next episode must be independent of the previous episode. These type of tasks are called *episodic tasks*.

However not all tasks are episodic, it is possible an RL-agent needs to solve some task that can continuo without a limit. These types of tasks are called *continuing tasks*. The expected return given by Eq. 3.2 can be problematic for these types of tasks, since the final time step would be at $T = \infty$. Thus it is possible the value it's trying to maximize is infinite as well. so to solve this problem the rewards can be *discounted*. In this approach the sum of discounted rewards, also known as the *discounted expected return*, shown in Eq. 3.3 is maximized.

his paragraph.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{3.3}$$

sing figure refer-
y this paragraph

It is possible to unify the expected return equations for the episodic and continuing tasks into a single one. For example, in the transition diagram for a continuos task shown below the environment transitions from states s_0 to s_1 and after its next action it ends up at a special absorbing state where all actions will result in the same state without any reward. This results in reward sequence $+1, +1, +1, 0, 0, ...$ By summing the whole infinite sequence we get the same result as summing the first T rewards, where $T = 3$ in this case. This is also true with discounting.

Thus Eq. 3.4 shows the unified equation for the (discounted) expected return for both episodic and continuing tasks. If the task at hand is continuing, then $T = \infty$, while for episodic tasks T will be some finite integer. It is possible to have $\gamma = 1$ for episodic tasks, but due to the reasons mentioned before the expected return must be discounted for continuing tasks, so $\gamma = 1$ and $T = \infty$ cannot true at the same time.

$$G_t \doteq \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \tag{3.4}$$

One important property of the expected return used in reinforcement learning is that it is recursive as demonstrated below.

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + ... \\ &= R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + ... \right) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \tag{3.5}$$

3.4. Policy and value functions

The policy is a mapping from the state of the environment to the probabilities of the agent selecting each action. If the policy at time t is π, then $\pi(a|s)$ is the probability of selecting action a if the current state is s.

The value of a state can be different depending on the policy of the RL-agent, since different policies will result in different actions and thus different states and rewards. So $v_\pi(s)$ is the expected return when starting at state s and following policy π. v_π can be defined by:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t | \, S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s \right], \text{ for all } s \text{ in } \mathcal{S} \tag{3.6}$$

For some problems it might be more suitable to look at the value of state-action pairs. Thus similar as to the state value above, the state-action value under policy π, denoted by $q_\pi(s,a)$, is defined by:

$$q_\pi(s,a) = \mathbb{E}_\pi \left[G_t | \, S_t = s, A_t = a \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s, A_t = a \right], \text{ for all } s \text{ in } \mathcal{S} \tag{3.7}$$

3.5. K-armed bandit problem

K-armed bandit problems will be used as examples in the following sections. These are simple learning problems where the agent is given the option to take k different action's. There is only one state and the episodes terminate after the agent has taken it's action. Furthermore the examples in this report use zero mean, unit variance normal distributions to select the true value $q_*(a)$ of the actions. The rewards given by the environment are then selected with $q_*(a)$ mean and unit variance normal distribution. Figure 3.1 shows an example reward distribution for a 5-armed bandit.

3.6. ϵ–greedy policies

The main difference between Reinforcement Learning and other types of machine learning is that RL uses training information that evaluates the actions taken by the agent instead of a set of correct actions. This means that in order to determine $Q_t(a)$ the agent needs to explore and try out as many actions as possible in order to find good behavior. To do this the agent can take random actions and learn from the rewards it gets back. This is called *exploration*.

However, the main goal of the agent is to maximize the rewards it receives from the environment over time. To achieve this goal the agent can choose to take the greedy actions. These are the actions with the highest value in $Q_t(a)$. This process of choosing the greedy action is known as *exploitation* since the agent is exploiting its knowledge to achieve its goal.

Unfortunately exploration and exploitation conflict with each other since in the former the agent wants to try out all possible actions, but only wants to perform a specific action if it's following a greedy policy. Thus a possible compromise is to use what is known as an ϵ-greedy policy. In these policies, the agent will choose a random action ϵ percentage of the time and the greedy action the rest of the time. This

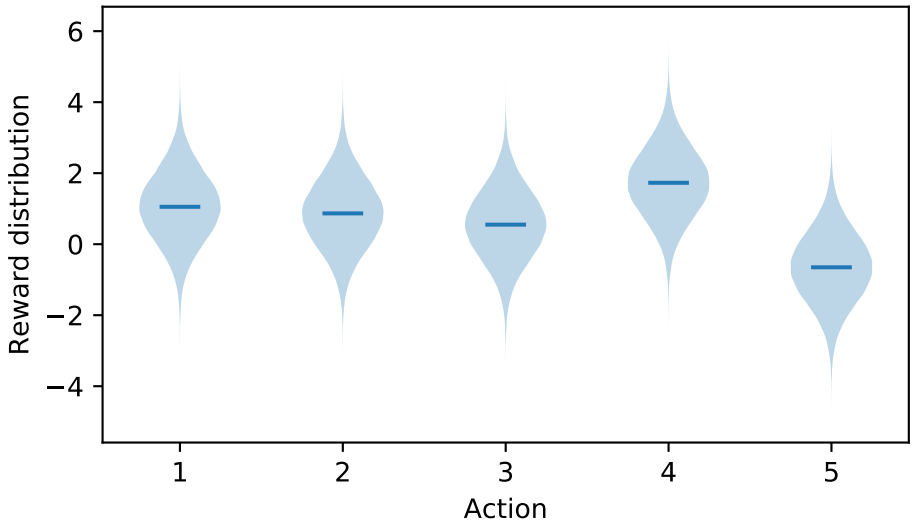


Figure 3.1: Example reward distribution for a 5-armed bandit problem.

this paragraph.

allows the agent to explore and try out new actions or old actions whose value might have changed, while also trying to achieve it's goal.

Since the agent is trying out new actions while following ϵ-greedy policies, it is able to learn faster and more than if it was following a greedy policy. To demonstrate this 2000 10-armed bandit problems are run for 1000 episodes. The average reward of all bandits are stored after each episode. Figure 3.2 shows the average reward and the percentage of agents that selected the optimal action for the greedy policy and two ϵ-greedy policies ($\epsilon = 0.01$ and $\epsilon = 0.1$). A full explanation of the learning method used can be found in the section **??**, but in short, the action value function $Q_t(a)$ is estimated by averaging the rewards received from the environment.

As it can be seen when using ϵ-greedy policies the agents learn faster and are thus able to receive higher rewards sooner. On the other hand, while following the greedy policy, the agents will quickly choose the first action they find with a reward higher than the initial action value and then perform that action on all episodes. As a results many of the agents are not able to find the optimal policy.

3.7. Dynamic Programming

Dynamic Programming (DP) refers to a collection of methods used to solve MDP problems. The main idea of DP and RL is the use of value functions to organize and structure the search for good policies. The optimal policies can easily be found once the optimal value functions v_* or q_* which satisfy the Bellman optimality equations shown below are known.

rence to the
n probability func-
anation in previ-
oter.

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_*(s')\right] \end{aligned} \tag{3.8}$$

$$\begin{aligned} q_*(s, a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a)\left[r + \gamma \max_{a'} q_*(S_{t+1}, a')\right] \end{aligned} \tag{3.9}$$

DP algorithms are found by turning the Bellman equations into assignment and, or update rules for improving approximations of the value functions.

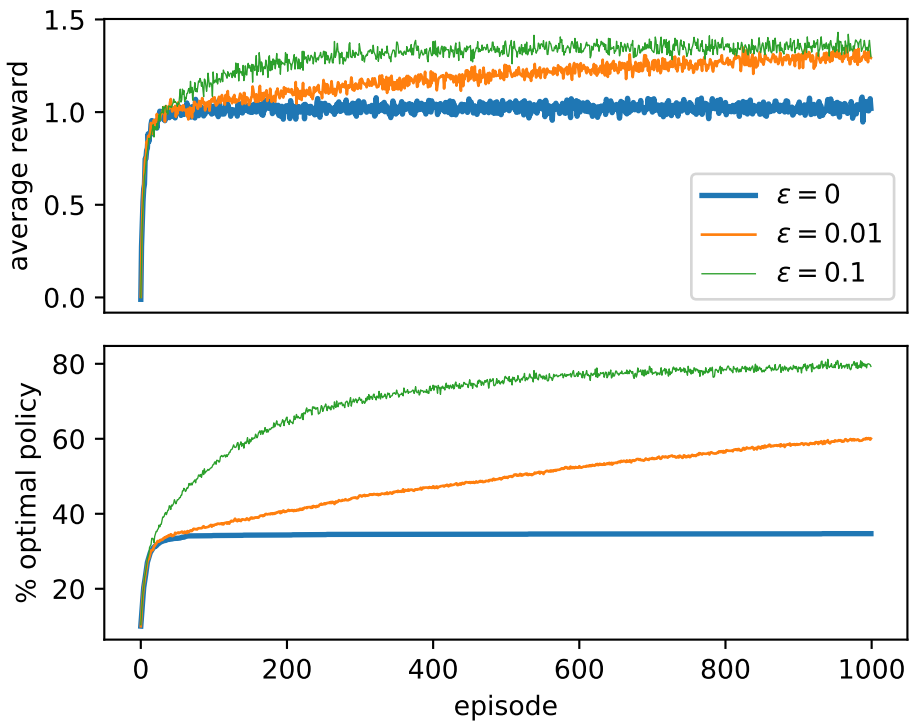


Figure 3.2: Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies.

3.7.1. Policy evaluation

The first step in DP is to compute the *state-value* function v_π for an arbitrary policy π. This is called *policy evaluation* or *prediction problem*.

This can be done by writing the state-value function v_π from eq. 3.6 in terms of the policy probability function $\pi(a|s)$ and the transition probability function $p(s', t|s, a)$. The policy probability function is the probability of action a being selected by the policy while being in state s. And the transition probability function gives the probability of the environment transitioning to state s' and giving reward r if action a is taken while in state s.

$$\begin{aligned} v_\pi(s) &\doteq \mathbb{E}_\pi\left[G_t|S_t = s\right] \\ &= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1}|S_t = s\right] \\ &= \mathbb{E}_\pi\left[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s\right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', t|s, a)\left[r + \gamma v_\pi(s')\right] \end{aligned} \tag{3.10}$$

If the environment's dynamics are known, then Eq. 3.10 is a system of $|\mathcal{S}|$ equations with $|\mathcal{S}|$ unknowns. Solving it would be tedious, but trivial. However, this is almost never the case, thus iterative solution methods provide a better approach. Consider a sequence of value functions $v_1, v_2, v_3, \ldots$, each mapping $\mathcal{S}^+$ to $\mathbb{R}$. The initial value function approximation may be chosen arbitrarily and new approximations are calculated using the Bellman equation for v_π as an update rule.

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}\left[R_{t+1} + \gamma v_k(S_{t+1})|S_t = s, A_t = a\right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_k(s')\right] \end{aligned} \tag{3.11}$$

The state-value function v_π is a fixed point for this update rule because the Bellman equation for v_π

assures us of equality in this case. It is also possible to show that the sequence v_k approaches v_π as $k \to \infty$. This algorithm is called the *iterative policy evaluation.*

To produce each approximation the iterative policy evaluation applies the operation to each state s and replacing the old value of s with a new one based on the immediate received reward and the old values of the successor states of state s. This operation is called an *expected update*.

One way to implement this algorithm is to keep two arrays, one to hold the old values and another for the new ones. This way the new values can be computed from the old ones, without changing the old ones. It is also possible to implement it with one array and update the values "in place", by immediately replacing the old value by the one. The in-place algorithm not only consumes less memory, but also usually converges faster than the two-array version. This is because the new values are used as soon as they are available.

3.7.2. Policy improvement

With the value function $v_\pi(s)$ for a deterministic policy π known, it is possible to check for each state, whether there is an action a which yields a larger value than the action taken by the current policy π. One way to do this, is for each state s, select an action a different from the action taken by the current policy ($a \neq \pi(s)$) and calculate the value if that action is taken first, followed by the current policy after that. The value for this behavior is:

$$\begin{aligned} q_\pi(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi\left(s'\right)\right] \end{aligned} \tag{3.12}$$

If this value is greater than that of the current policy, then a better action has been found. It can therefore be expected that a policy where the new action a is taken whenever state s is encountered is a better policy than the current policy. Thus updating the policy π with the new action will result in a new policy that is as good as or better than the current policy.

Repeating this process for all state-action combinations, it is possible to find a new greedy policy π', given below. Where $\arg\max_a$ denotes the value of a which maximizes the expression that follows.

$$\begin{aligned} q_\pi(s,a) &\doteq \arg\max_a \; q_\pi(s,a) \\ &= \arg\max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right) | S_t = s, A_t = a\right] \\ &= \arg\max_a \; \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi\left(s'\right)\right] \end{aligned} \tag{3.13}$$

The process of making a new policy that improves over the original one using the values of the original policy is called *policy improvement*. In the case where the new policy π' is as good than the old policy π, Then $v_{\pi'} = v_\pi$ and from there is follows that for all $s \in \mathcal{S}$:

$$\begin{aligned} v_{\pi'}(s) &= \max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_{\pi'}(s')\right] \end{aligned} \tag{3.14}$$

This is the same as the Bellman optimality equation (3.8) and thus $v_{\pi'}(s)$ is the optimal value function $v_*(s)$. So policy improvement will always give a better policy, except when the policy is already optimal.

Policy improvement can also be applied to stochastic policies. These are policies where each action is given a probability $\pi(a|s)$ to occur. If there are ties between actions, that is, multiple actions that result in the same maximum value, then each of these action can be given a part of the probability to occur, as long as any non-optimal action is given zero probability.

3.7.3. Policy iteration

Using policy improvement a policy, π, can be improved using v_π to yield a better policy, π'. With this new policy a new *state-value* function v'_π can be evaluated from π', which results a new improved policy π''. This process can be repeated indefinitely.

Each new policy in guaranteed to be in improvement over the previous one unless it's already the optimal policy. Since a finite MDP only had a finite number of policies, this process must converge to the optimal policy and the optimal value function in a finite number of iterations.

This way of finding the optimal policy it called policy iteration.

3.7.4. Generalized Policy Iteration

In Policy Iteration the Policy Improvement and Policy Evaluation processes alternate, each completing before the other one. However this is not necessary, as long both processes continue to update all states, the ultimate result is almost always the same, convergence to the optimal value function and optimal policy. The term Generalized Policy Iteration (GPI) is used to refer the general idea of letting the Policy Improvement and Policy Evaluation processes interact independent of the granularity and other details of the other processes.

3.8. Monte Carlo Methods

It is not always possible to have a model fully describing the system for which the optimal policy is being discovered. This is where Monte Carlo methods come into play. Unlike Dynamic Programming, Monte Carlo methods require only *experience*, sample sequences of states, actions and rewards or simulated interaction with the environment. Thus no prior knowledge of the dynamics are needed to determine the optimal policy. These methods allow learning from simulated experience, which would require a model, but this model only needs to generate sample transitions and not the complete probability distribution of all possible transitions.

Just like the k-armed bandit problem demonstrated in Section **??**, Monte Carlo methods work by averaging the returns for state—action pairs. The main difference is that there are multiple states, each acting like a different, interrelated bandit problems, that is, the return from taking an action in one state depends on the actions taken in future states in the same episode.

Since the agent is learning and changing its policy throughout the process, the action selection is constantly changing and thus the problem is nonstationary from the point of view of the earlier state. To handle the nonstationarity, the idea of Generalized Policy Iteration (GPI) is adapted. In DP the value functions are computed directly from the knowledge of the MDP. With Monte Carlo methods however the value functions are *learned* from sample returns with the MDP. The value functions and their policies still interact in the same way.

As previously mentioned, the value of a state ($v_\pi(s)$) is the expected return, the expected cumulative future discounted reward, starting from that state. The obvious way to estimate it from experience is to average all returns after visiting that state. As more returns are observed, the average will converge to the expected value. This is the underlying idea of all Monte Carlo methods.

If no model is available, then it might be better to estimate *action* values (the values of state—action pairs, $q_\pi(s, a)$). With a model, state values alone are sufficient to determine a policy, since the agent would simply have to look ahead one step and see which action results in the highest value. This is not possible without a model. The values of each action must be estimated, since the agent doesn't know to witch state each action will lead to.

3.9. Temporal Difference

Temporal-Difference (TD) learning is one of the main elements of reinforcement learning. It is a combination of ideas from Monte Carlo methods and DP. Just as with Monte Carlo methods, Temporal-Difference (TD) methods are able to learn from experience without the need of a model of the environment's dynamics, and just as with DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome.

3.9.1. TD Prediction

Both TD and Monte Carlo methods use experience to solve the prediction problem. Given some experience following policy π, both methods update their estimate V of v_π for non-terminal states that occurred during that experience. Monte Carlo methods wait until the return G_t after visiting state S_t is known. A simple every-visit Monte-Carlo method is shown in Equation 3.15.

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t - V(S_t)\right] \tag{3.15}$$

Monte-Carlo methods need to wait until the end of an episode since G_t is only known then. TD methods only need to wait until the next iteration by estimating G_t using the observed reward R_{t+1} and the estimated value $V(S_{t+1})$. Thus just like with DP, TD methods updates are based in part on existing estimates. These types of methods are known as bootstrapping methods.

Equation 3.16 shows the simplest TD method. This method is called TD(0) or one-step TD, because it's a special case of the TD(λ) and n-step TD methods.

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right] \tag{3.16}$$

The quantity within the bracket is sort of an error, it's the difference between the estimated value of S_t and the better estimate $R_{t+1} + \gamma V(S_{t+1})$. This quantity is known as the TD error and can be found in various forms throughout reinforcement learning.

3.9.2. Sarsa: On-policy TD Control

In order to use TD prediction for the control problem the pattern of Generalized Policy Iteration (GPI) can be used, only this time using TD methods for the evaluation or prediction part. Approaches fall under two classes, on-policy and off-policy. This section focuses on an on-policy TD control method.

The first step is to learn an action-value function rather than a state-value function. For an on-policy method the estimated $q_\pi(s, a)$ is made for the behavior of the current policy. This can be done essentially the same as the TD method described above.

uple

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right] \tag{3.17}$$

This update is done after every iteration from a non-terminal state S_t. If S_{t+1} is a terminal state, then $Q(S_{t+1}, A_{t+1})$ is defined as zero. This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state-action to another. This quintuple gives rise to the name Sarsa for the algorithm.

3.9.3. Q-learning: Off-policy TD control

Another option is Q-learning, an off-policy TD control method. In this case the learned action-value function, directly approximates q_*, the optimal action-value function.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{3.18}$$

The advantage of this method is that it drastically simplifies the analysis of the algorithm and enables early convergence proofs. The only thing that is required is that all pairs continue to be updated. The disadvantage is that since the action-value function learned is independent of the policy, the agent doesn't take into account the possibility of it taking a non-optimal action.

The following example is used to demonstrate the difference between on and off policy methods. Figure 3.3 shows a grid world. The agent's task to go from the start (S) block to the goal (G). However if the agent steps on the cliff, the episode will end, it receives a -100 reward and is then send back to the start. A -1 reward is given to agent each step until it has reached the goal.

captions

200 agents are simulated for 500 episodes to demonstrate the difference between on and off policy learning methods. They all follow ϵ-greedy policies with $\epsilon = 0.1$, but half use Sarsa and the other half

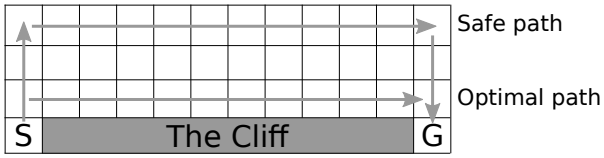


Figure 3.3: Example reward distribution for a 5-armed bandit problem.

Q-learning. The Q-learning agents learn the optimal policy and thus travel right along the edge of the cliff. However since the agents may at any point take a step into any random direction, it is better to take the safer route further away from the edge. Since the Sarsa agents learn the roundabout policy, this is the path they learn to take. Figure 3.4 show the results of these simulations.

However once the agents have been trained and $\epsilon$is set to zero, the Q-learning agents outperform the Sarsa agents. With these new greedy policies the Q-learning agents will always receive a sum of rewards of -12 while the Sarsa agents receive -16 thanks to the longer path. If $\epsilon$is gradually reduced to zero, then both methods converge to the optimal policy.

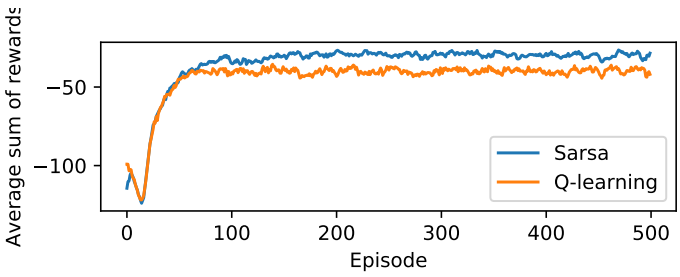


Figure 3.4: Example reward distribution for a 5-armed bandit problem.

3.9.4. n-step Bootstrapping

Consider the expected return used with Monte Carlo methods shown below. It's based on the rewards starting from time t until the end of the episode. Thus while the update does include information throughout a long period of time, it's only possible to update the values after the episode has ended.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... + \gamma^{T-t-1} R_T \tag{3.19}$$

In contrast the expected return for one-step TD only uses single reward after one time-step. For many applications it's desirable to update as often as possible, but bootstrapping works best over a period of time after significant and recognizable change has occurred. With one-step TD methods these two lengths of time are the same, so a compromise must be made.

$$G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1}) \tag{3.20}$$

While both of these methods are effective in solving MDP problems, the best solutions are usually somewhere in between. n-step Bootstrapping methods provide a spectrum of methods with Monte Carlo in one end and one-step TD on the other by enabling bootstrapping to occur over multiple time steps.

An example family of n-step Bootstrapping methods are n-step TD Prediction methods. For these methods the n-*step return* is as follows,

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + ... + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n}) \tag{3.21}$$

As it is clear from the Equation 3.21, the expected return uses the reward of multiple steps, thus allowing the value function to be updated on each time-step based on information collected over a longer period of time.

3.10. Approximate solutions

Up until now all methods discussed used tables to store the learned value functions and policies. However these methods are not practical for many of the tasks where Reinforcement Learning (RL) is applied due these having large and continuos state spaces.

The problem with large state spaces is not just the memory needed to store the table, but also the time and amount of data needed to fill in these tables accurately. For many of these tasks the states encountered will never have been seen before, thus to make decisions it is necessary to generalize from previous encounters with different states that are similar to the current one. In other words the key issue is that of *generalization*. How can experience with a limited subset of states be generalized to produce a good approximation of a much larger subset?

Fortunately, generalization from examples has been extensively studied, and thus the problem to be solved is combining existing generalization methods with RL methods. The type of generalization methods required are often called *function approximation* because it takes in samples of the target function and creates an approximation of the entire function.

3.10.1. Stochastic Gradient Decent methods

Stochastic Gradient Decent (SGD) methods are a class of function approximation learning methods well suited for the field of Reinforcement Learning (RL). In these methods the weight vector is a real valued column vector with a fixed number of elements, $\mathbf{w} = (w_1, w_2, ..., w_n)^T$, and the approximate function $\hat{v}(s, \mathbf{w})$ is differentiable w.r.t. all weights in $\mathbf{w}$ at all states $s \in \mathcal{S}$.

The goal is to update the weights in $\mathbf{w}$ after each observation such that the approximate function $\hat{v}(s, \mathbf{w})$ better approximates the real function $v_\pi(s)$. This can be done by updating the weights in the direction that would reduce the error the most. A definition of this error is thus needed, one possibility is to use the *Mean Squared Value Error* $\overline{\text{VE}}$ shown below [10].

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w})\right]^2 \tag{3.22}$$

Here is $\mu(s)$ the *state weight distribution*, it is used to indicate which states are more important. This may be necessary since the number of weights in $\mathbf{w}$ can be significantly smaller than the total number of states, thus prioritizing certain states may be needed.

The following update rule for the weights in $\mathbf{w}$ may be defined, using $\overline{\text{VE}}$ and assuming the probability for each state to appear is uniformly distributed.

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t - \frac{1}{2}\alpha \nabla \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w})\right]^2 \tag{3.23}$$

$$= \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w}) \tag{3.24}$$

SGD methods are *Gradient Decent* because the updates to the weight vector $\mathbf{w}$ are proportional to the negative gradient of each sample's error. This is the direction where error falls the fastest. Gradient decent methods are called *stochastic* when the updates are done using single samples which may have been selected stochastically. In these cases the updates apply small corrections to the weights and over the course of may iterations it has the effect of reducing the error.

The real value function $v_\pi(s)$ is in practice often not known. Thus the Stochastic Gradient Decent (SGD) may be generalized by substituting it with U_t. This not the true value, $v_\pi(S_t)$, but an approximation of it. It may for example be a noise-corrupted version of $v_\pi(S_t)$, or one of the bootstrapping targets for $\hat{v}$ (eg. G_t, $G_{t:t+n}$, etc).

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[U_t - \hat{v}\left(S_t, \mathbf{w}\right)\right] \nabla \hat{v}\left(S_t, \mathbf{w}\right) \tag{3.25}$$

3.10.2. Adam

In SGD the step size parameter, α, has a significant effect on the learning performance. If it's too small, it will learn slowly, if it's too large the target function will not be properly approximated. The step size is also kept constant throughout the learning process and the same value is used to update all parameters in the weight vector, $\mathbf{w}$. This is not ideal, since it may in some cases be beneficial to have separate learning rates for each weight and to update these rates as the agent explores the environment and learns.

Adam is an extension to SGD that uses estimates of the gradient's average 1st moment, better known as the mean, and the gradient's average 2nd moment, better known as the uncentered variance, to adapt the learning rate of each individual parameter in the weight vector.

Algorithm 1: Adam algorithm. Good default values for the hyperparameters are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element wise. With β_1^t and β_2^t being β_1 and β_2 to the power of t[6].

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\mathbf{w})$: Stochastic objective function with parameters $\mathbf{w}$
Require: $\mathbf{w}_0$: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize time step)
while $\mathbf{w}_t$ *not converged* **do**
  $t \leftarrow t + 1$
  $\mathbf{g_t} \leftarrow \nabla_w f_t(\mathbf{w}_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
  $\mathbf{m}_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t$ (Update biased first moment estimate)
  $\mathbf{v}_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_t^2$ (Update biased second moment estimate)
  $\widehat{\mathbf{m}_t} \leftarrow \mathbf{m}_t / (1 - \beta_1^t)$ (Compute biased-corrected first moment estimate)
  $\widehat{\mathbf{v}_t} \leftarrow \mathbf{v}_t / (1 - \beta_2^t)$ (Compute biased-corrected second raw moment estimate)
  $\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \alpha \cdot \widehat{\mathbf{m}_t} / (\sqrt{\widehat{\mathbf{v}_t}} + \epsilon)$ (Update parameters)
end
return $\mathbf{w}_t$ (Resulting parameters)

The algorithm updates exponential moving averages of the gradient, $\mathbf{m}_t$, and the squared gradient, $\mathbf{v}_t$, where the hyperparameters β_1 and β_2 control the exponential decay. These moving averages are themselves estimates of the 1st and 2nd average moments. These vectors are initialized at zero, thus the estimates are biased towards zero, specially during the initial timesteps and when the exponential decays are close to one. This initialization bias is then counteracted yielding the bias-corrected estimates $\widehat{\mathbf{m}_t}$ and $\widehat{\mathbf{v}_t}$.

An important property of the update rule is its careful choice of stepsizes. Assuming $\epsilon = 0$, the effective step taken by each parameter at timestep t is $\Delta_t = \alpha \cdot \widehat{\mathbf{m}_t} / \sqrt{\widehat{\mathbf{v}_t}}$. This effective stepsize has two upper bounds: $|\Delta_t| \leq \alpha \cdot (1 - \beta_1) / \sqrt{1 - \beta_2}$ in case $(1 - \beta_1) > \sqrt{1 - \beta_2}$, and $|\Delta_t| \leq \alpha$ otherwise.

The first case only happens in severe cases of sparsity. When a gradient has been zero for all timesteps except the current one. For less sparse cases, the effective stepsize is smaller. When $(1 - \beta_1) = \sqrt{1 - \beta_2}$ then $|\widehat{\mathbf{m}_t} / \sqrt{\widehat{\mathbf{v}_t}}| < 1$ therefore $\Delta_t < \alpha$. In most cases $\widehat{\mathbf{m}_t} / \sqrt{\widehat{\mathbf{v}_t}} \approx \pm 1$ since $|\mathbb{E}[\mathbf{g}] / \sqrt{\mathbb{E}[\mathbf{g}^2]}| \leq 1$. Therefore the effective stepsize is bounded by the stepsize hyperparameter, α. This can be understood as establishing a *trust region* around the current weight parameter values, beyond which the current gradient estimate does not provide enough information. This makes it easier to choose a value for α since it only indicates an upper bound for the stepsize.

The $\widehat{\mathbf{m}_t} / \sqrt{\widehat{\mathbf{v}_t}}$ may be seen as some sort of a Signal to Noise Ratio (SNR). With a smaller SNR the effective stepsize will be closer to zero. This property is desirable, since a smaller SNR indicates a greater uncertainty on whether the direction of $\widehat{\mathbf{m}_t}$ corresponds to the direction of the true gradient.

For example, the SNR becomes close to 0 towards the optimum, thus reducing the effective stepsize, resulting in a form of automatic step size annealing. The effective stepsize is also invariant of the scale of the gradients. Scaling the gradient $\mathbf{g}$ with a factor c will scale $\widehat{\mathbf{m}_t}$ by c and $\widehat{\mathbf{v}_t}$ by c^2, which cancel out in the SNR.

3.10.3. Artificial Neural Networks

Artificial Neural Networks (ANNs) are widely used for nonlinear function approximation in the field of machine learning and beyond. Deeply-layered ANNs have successfully been used in the Reinforcement Learning to solve complex problems.

An ANN consists of interconnected units which have a similar function to a neuron in a nervous system. Figure 3.5 shows a simple Feed Forward Neural Network (FFNN) with three inputs, three hidden layers and two outputs. The layers are connected to with links, with each link having a real-valued weight associated with it. The links are comparable to synaptic connections in a nervous system and the weight are roughly comparable to the efficacy of the synaptic connections.

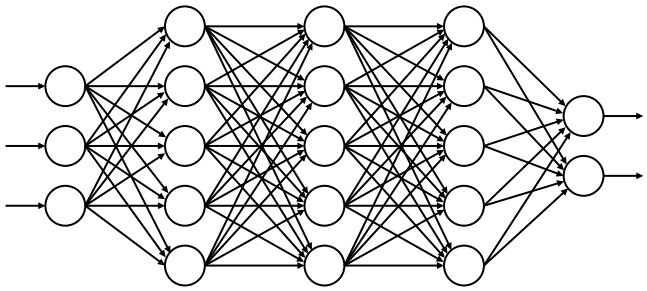


Figure 3.5: Artificial Neural Network[10].

Each one of the units computes its output by first calculating a weighted sum of the inputs and then applies a non-linear function to the result. This non-linear function is called an *activation function*. The activation function is often S-shaped, or sigmoid, for example a hyperbolic tangent, but sometimes a rectifier non-linearity, $f(x) = \max(0, x)$, or step function are also used. It's often useful for different layers to have different types of activation functions.

A FFNN with no hidden layers is only able to approximate linear functions, however if a single hidden layer with a large enough number sigmoid elements is added, then any continuos function may be approximated within a range of input values with any degree of accuracy[3]. This is also true for other non-linear functions that satisfy certain conditions, however non-linearity is essential. Non-linear activation functions are equivalent to a network without hidden layers, since linear functions of linear functions are linear.

Even though a single hidden layer is enough to approximate any function, both practice and theory show that for the complex functions needed in many artificial intelligent tasks it's often easier or may even require abstractions which are hierarchical compositions of many layers of lower-level abstractions, that is, abstractions produced by deep architectures such as ANNs with many hidden layers [1, 10].

3.10.4. LSTM Neural Networks

The network shown in Figure 3.5 is called a Feed Forward Neural Network because it contains no loops, that is, the output of a unit cannot influence the its own input. Networks with loops are called Recurrent Neural Networks (RNNs). An RNN can use the feedback loops to store representations of inputs from recent events. The main advantage of RNNs are the ability to exhibit temporal dynamic behavior, this property allows agents to process sequences of variable length, learn how to behave when there are long intervals of time between events and learn policies for non-Markovian environments.

The Long Short-Term Memory (LSTM) architecture is a type of RNN that is quite popular and has been in use for many years. An LSTM unit consists of a cell, an input gate, an output gate and a forget gate. The cell remembers values for arbitrary time intervals and the three gates regulate the flow of information in and out of the cell[4]. One or more LSTM hidden layers containing an array of these

cells may then be used together with feed forward layers to build a deep neural network exhibiting the properties of an RNN.

3.11. Policy Gradient Methods

So far the methods discussed select actions based on estimated (state-)action values, however this is not the only option. Instead of learning the value estimates and deriving a policy out of the resulting value functions, it is also possible to directly learn a *parametrized policy* which can choose actions without having to consult a value function.

Policy gradient methods are a class of learning methods that learn the policy parameters, $\boldsymbol{\theta}$, based on the gradient of some performance measure, $J(\boldsymbol{\theta})$, with respect to the policy parameters. These methods try to maximize the performance, so the updates approximate gradient accent in J.

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} + \alpha \cdot \widehat{\nabla J(\boldsymbol{\theta_t})} \tag{3.26}$$

where α is a positive step-size parameter and $\widehat{\nabla J(\boldsymbol{\theta_t})}$ is a stochastic estimate who's expectation is the gradient of the performance measure with respect to $\boldsymbol{\theta_t}$.

An advantage of using policy gradient over ϵ-greedy action selection is that the approximate policy can approach a deterministic policy if the optimal policy is deterministic, whereas ϵ-greedy action selection will always have an ϵ probability of selecting a random action.

Similarly if the optimal policy is stochastic or in cases where multiple actions have similar values, then policy gradient methods are able to converge towards stochastic policies who's probability distribution approximate the distribution of the optimal policy. In contrast action-value methods are unable to find optimal stochastic policies.

In problems with significant function approximation, the best policy may be stochastic even for deterministic problems. With imperfect information the optimal policy may be to perform two or more different actions with specific probabilities, for example when bluffing in Poker.

This is specially helpful for continuos state and action spaces. Since, as discussed in Section 3.10, for these kind of systems it is usually not practical to store, process and learn policy and value functions based on tables. Instead these functions are usually approximated.

3.11.1. Policy parameterization

In policy gradient methods the policy may be parametrized in any way so long as the policy function, $\pi(a|s,\ \boldsymbol{\theta})$, is differentiable with respect to the policy parameters, $\boldsymbol{\theta}$.

A method of parameterization in case of discreet and small action spaces is to form parametrized numerical preferences $h(s, a, \boldsymbol{\theta}) \in \mathbb{R}$ for each state-action pair. The actions with the highest preference can then be given the highest probability of being selected by, for example, using an exponential softmax distribution,

$$\pi(a|s, \boldsymbol{\theta}) = \frac{\exp(h(s, a, \boldsymbol{\theta}))}{\sum_b \exp(h(s, b, \boldsymbol{\theta}))} \tag{3.27}$$

The denominator is needed such that the probabilities in each state sum to one, thus allowing the preference to be parametrized using any function, whether it is a table, a linear function using feature vectors or a deep neural network.

For problems with large or continuos action spaces it may be better to learn the statistics of the probability distribution instead of the preference of each individual action. For example, the actions may be chosen using a normal (Gaussian) distribution. In this case the mean, μ, and standard deviation, σ, of the distribution may be parametrized using any function approximation method, with the only limitation being that the function for the standard deviation must always be positive.

$$\pi\left(a|s,\boldsymbol{\theta}\right)=\frac{1}{\sigma(s,\boldsymbol{\theta})\sqrt{2\pi}}\exp\left(-\frac{(a-\mu(s,\theta))^2}{2\sigma(s,\theta)^2}\right) \tag{3.28}$$

3.11.2. Policy gradient theorem

The goal with MDP problems is to find a policy which maximizes the sum of rewards the agent receives. The value function approximates an expectation of the sum of rewards, thus the perform measure, $J\left(\boldsymbol{\theta}\right)$, can be assumed as being approximately equal to the true value function under current policy π_θ.

$$J\left(\boldsymbol{\theta}\right)\doteq v_{\pi_\theta}\left(s\right) \tag{3.29}$$

The problem with finding a change in the policy parameters that ensures improvement is that the performance depends on both the action selection and the distribution of states. At a given state, estimating the effect on the actions, and thus the reward, is relatively straightforward. However the effect of the policy on the state distribution depends on effects from the environment which are typically unknown.

Fortunately, the *policy gradient theorem* provides an analytical expression that is proportional to the gradient of the performance with respect to the policy parameters, which doesn't involve the derivative of the state distribution[10].

$$\nabla J\left(\boldsymbol{\theta}\right)\propto\sum_s\mu\left(s\right)\sum_a q_\pi\left(s,a\right)\nabla_\theta\pi\left(a|s,\boldsymbol{\theta}\right) \tag{3.30}$$

3.11.3. REINFORCE

Recall from Equation 3.26 the overall strategy for policy gradient methods requires an estimate of the gradient of the performance measure with respect to the policy parameters. This estimate only needs to be proportional to the real gradient, since any constant of proportionality can be absorbed by the step size α, which is otherwise arbitrary. The policy gradient theorem (Equation 3.30) gives an expression for such an estimate.

The right-hand side of the policy gradient theorem is a sum over the states weighted by how often the states occur under the target policy π. If π is followed, then the states will be encountered in these proportions. Thus,

$$\nabla J\left(\boldsymbol{\theta}\right)\propto\sum_s\mu\left(s\right)\sum_a q_\pi\left(s,a\right)\nabla_\theta\pi\left(a|s,\boldsymbol{\theta}\right) \tag{3.31}$$

$$=\mathbb{E}_\pi\left[\sum_a q_\pi\left(S_t,a\right)\nabla_\theta\pi\left(a|S_t,\boldsymbol{\theta}\right)\right] \tag{3.32}$$

The remaining part of the expectation is a sum over the actions. If each term was weighted by the probability of selecting the actions, that is by, $\pi\left(a|S_t,\theta\right)$, then it is possible to replace a by the sample action A_t. This can be done by multiplying and dividing by this probability.

$$\nabla J\left(\boldsymbol{\theta}\right)=\mathbb{E}_\pi\left[\sum_a\pi\left(a|S_t,\theta\right)q_\pi\left(S_t,a\right)\frac{\nabla_\theta\pi\left(a|S_t,\boldsymbol{\theta}\right)}{\pi\left(a|S_t,\theta\right)}\right] \tag{3.33}$$

$$=\mathbb{E}_\pi\left[q_\pi\left(S_t,A_t\right)\frac{\nabla_\theta\pi\left(A_t|S_t,\boldsymbol{\theta}\right)}{\pi\left(A_t|S_t,\theta\right)}\right] \quad \text{(replacing } a \text{ by sample } A_t\sim\pi\text{)} \tag{3.34}$$

$$=\mathbb{E}_\pi\left[G_t\frac{\nabla_\theta\pi\left(A_t|S_t,\boldsymbol{\theta}\right)}{\pi\left(A_t|S_t,\theta\right)}\right] \quad \text{(Because } \mathbb{E}_\pi\left[G_t|S_t,A_t\right]=q_\pi\left(S_t,A_t\right)\text{, Equation 3.7)} \tag{3.35}$$

Substituting this expression into Equation 3.26 yields the update rule,

$$\boldsymbol{\theta}_{\mathbf{t+1}} = \boldsymbol{\theta}_{\mathbf{t}} + \alpha G_t \frac{\nabla_\theta \pi\left(A_t|S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t|S_t, \theta\right)} \tag{3.36}$$

This is the REINFORCE algorithm. Note that REINFORCE uses the complete return from t until the end of the episode. In this sense REINFORCE is a Monte Carlo algorithm and is thus only well defined for episodic problems.

Each update is proportional to the product of the return, G_t, and a vector in the direction that most increases the probability of taking A_t on future visits to S_t. This vector is the *eligibility vector*. The eligibility vector is usually written in the more compact form $\nabla_\theta \ln \pi\left(A_t|S_t, \boldsymbol{\theta}\right)$, using $\nabla \ln x = \frac{\nabla x}{x}$. Thus the REINFORCE algorithm may also be written as,

$$\boldsymbol{\theta}_{\mathbf{t+1}} = \boldsymbol{\theta}_{\mathbf{t}} + \alpha G_t \nabla_\theta \ln \pi\left(A_t|S_t, \boldsymbol{\theta}\right) \tag{3.37}$$

3.11.4. REINFORCE with baseline

The policy gradient theorem can be generalized by including a comparison of the action value to an arbitrary baseline $b(s)$.

$$\nabla J\left(\boldsymbol{\theta}\right) \propto \sum_s \mu\left(s\right) \sum_a \left(q_\pi\left(s, a\right) - b(s)\right) \nabla_\theta \pi\left(a|s, \boldsymbol{\theta}\right) \tag{3.38}$$

This baseline can be any function, even a random variable, as long as it does not vary with the action[10]. Following a similar procedure as in the previous section yields the following update rule for REINFORCE with baseline,

$$\boldsymbol{\theta}_{\mathbf{t+1}} = \boldsymbol{\theta}_{\mathbf{t}} + \alpha\left(G_t - b(S_t)\right) \frac{\nabla_\theta \pi\left(A_t|S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t|S_t, \theta\right)} \tag{3.39}$$

The baseline leaves the expected value of the update unchanged, however it can have a large effect on its variance and thus the learning rate. An appropriate choice for the baseline is an estimate of the state-value function $\hat{v}\left(S_t\right)$[10]. In this case, the term with the expected return and baseline value function is known as the expected advantage function.

$$A_t = G_t - \hat{v}(S_t) \tag{3.40}$$

Since REINFORCE is a Monte Carlo method for learning the policy parameters θ, it seems appropriate to use the similar Stochastic Gradient Decent (SGD) function approximation learning method shown in Section 3.10.1 to learn the value function with weights $\mathbf{w}$.

3.11.5. Actor-Critic

Methods that learn both the policy and value functions are called *actor-critic* methods, where the actor refers to the learned policy, and the critic to the learned value function. However REINFORCE with baseline is not considered an actor-critic method because the state-value function is only used as a baseline and not a critic. That is, it is not used for *bootstrapping*, the process of updating the value estimate of a state using the estimated values of future states, such as with the Temporal-Difference (TD) methods found in Section 3.9.

Consider the one-step actor-critic methods. These methods replace the complete return used in REINFORCE with baseline with the one-step return. Expanding these methods to include n-step bootstrapping is then straightforward.

$$\boldsymbol{\theta}_{\mathbf{t+1}} = \boldsymbol{\theta}_{\mathbf{t}} + \alpha \left(G_{t:t+1} - \hat{v}\left(S_t, \mathbf{w}\right)\right) \frac{\nabla_\theta \pi\left(A_t|S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t|S_t, \theta\right)} \tag{3.41}$$

$$= \boldsymbol{\theta}_{\mathbf{t}} + \alpha \left(R_{t+1} + \gamma V_t(S_{t+1}) - \hat{v}\left(S_t, \mathbf{w}\right)\right) \frac{\nabla_\theta \pi\left(A_t|S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t|S_t, \theta\right)} \tag{3.42}$$

$$= \boldsymbol{\theta}_{\mathbf{t}} + \alpha \delta_t \frac{\nabla_\theta \pi\left(A_t|S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t|S_t, \theta\right)} \tag{3.43}$$

3.11.6. PPO

A limitation of standard policy gradient methods is that it is hard to choose step sizes when updating the policy. This is in part because a bad update step is quite damaging since it may reduce the visitation distribution around the state. Stepping to far results in a bad policy, which leads to new data being collected under the bad policy, which can cause performance collapse. Another issue with standard policy gradient methods is that these methods have low sample efficiency, only one gradient step is performed per environment sample[8, 9].

So, how can the biggest improvement step on the policy be taken without over stepping and causing performance collapse? This is the question that methods such as Trust Region Policy Optimization (TRPO), and Proximal Policy Optimization (PPO) seek to answer.

In TRPO a surrogate objective function is maximized. This function is an approximation of the expected advantage function of the current or new policy. It is the expected advantage from the old policy being used when the sample was taken and then calibrated using the probability ratio between the new and old policies. This approximation becomes less accurate, as the two policies deviate from each other.

This objective function is then constrained by the KL-divergence of the new and old policies. The KL-divergence function measures the difference between two probability distributions. This problem is then solved using the conjugate gradient algorithm after making a linear approximation of the objective function and a quadratic approximation of the constraint [8].

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \mathbb{E}_{\mathbb{t}}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t\right] \\ \text{subject to} \quad & \mathbb{E}_{\mathbb{t}}\left[\overline{KL}\left[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)\right]\right] \leq \delta \end{aligned} \tag{3.44}$$

The theory justifying TRPO actually suggests using a penalty instead of a constraint as shown in Equation 3.45. In this case the objective function computes the maximum KL-divergence over the states and subtracts it from the expected advantage estimate,forming a lower or pessimistic bound on the performance of the new policy, π. This lower bound function can then be used to optimize the policy for maximum rewards using the Minorize-Maximization (MM) algorithm shown in Figure 3.6. MM works by iteratively maximizing a lower bound function which locally approximates the target function.

$$\underset{\theta}{\text{maximize}} \quad \mathbb{E}_{\mathbb{t}}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t - \beta\, \overline{KL}\left[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)\right]\right] \tag{3.45}$$

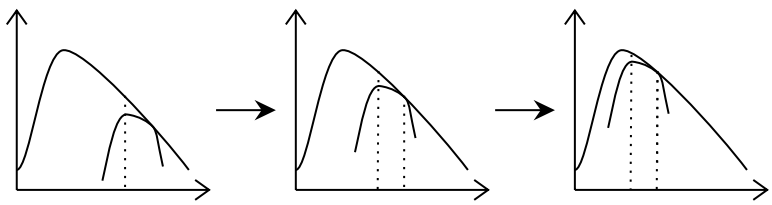


Figure 3.6: Minorize-Maximization algorithm

The reason TRPO uses a hard constraint instead of the penalty is because it is difficult to choose a value for β that works well between problems or even within the same problem. However PPO provides new

alternate surrogate objective functions that work around this issue. Despite PPO not being as good as TRPO in determining an appropriate size for the trust region, it is much simpler to implement and less computationally intensive. Benchmarks show that PPO is able to perform as good as or even better than other state-of-the-art methods[9].

There are two variants of PPO. PPO with adaptive KL penalty and PPO with clipped objective. This thesis only uses PPO with clipped objective, thus the first variant is left out.

Let $r_t(\boldsymbol{\theta})$ be the probability ratio $r_t(\boldsymbol{\theta}) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. Then the policy surrogate objective function proposed by PPO is as follows.

$$L_t^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbb{t}}\left[\min(r_t(\boldsymbol{\theta})A_t, \text{clip}(r_t(\boldsymbol{\theta}), 1-\epsilon, 1+\epsilon)A_t)\right] \tag{3.46}$$

Where ϵ is a hyperparameter, in the original paper the value of $\epsilon = 0.2$ was chosen based on benchmarks[9]. The first term inside the $\min(...)$ is the same as the surrogate objective function used in TRPO (Equation 3.44). The second term clip$(r_t(\boldsymbol{\theta}), 1-\epsilon, 1+\epsilon)A_t$ modifies the surrogate objective by clipping the probability ratio, thus removing the incentive for moving r_t outside of the interval $[1-\epsilon, 1+\epsilon]$

If a neural network architecture that shares parameters between the policy and value functions is being used, then a loss function that combines the policy surrogate objective function and a value function error term must be used. An entropy bonus may also be added to encourage exploration. Combining these terms results in the following objective function[9].

$$L_t^{CLIP+VF+S}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbb{t}}\left[L_t^{CLIP}(\boldsymbol{\theta}) + c_1 L_t^{VF}(\boldsymbol{\theta}) + c_2 S[\pi_\theta](s_t)\right] \tag{3.47}$$

Here are c_1 and c_2 coefficients, S the entropy bonus and L_t^{VF} is the squared-error loss of the value function estimate. This objective function can then be optimized on each iteration using SGD or Adam. This allows each environment sample to be used for multiple policy updates, thus resulting in a better sample efficiency than standard gradient decent methods.

4

Conclusion

5

Proposed thesis research

Bibliography

[1] Yoshua Bengio. *Learning deep architectures for AI*. Now Publishers Inc, 2009.

[2] John T Betts. *Practical methods for optimal control and estimation using nonlinear programming*. SIAM, 2010.

[3] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

[5] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017.

[6] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. URL `https://arxiv.org/abs/1412.6980`.

[7] Anil V Rao. A survey of numerical methods for optimal control. *Advances in the Astronautical Sciences*, 135(1):497–528, 2009.

[8] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015. URL `https://arxiv.org/abs/1502.05477`.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[11] William Tyrrell Thomson. *Introduction to space dynamics*. Courier Corporation, 2012.